

Supplementary Materials: Per-Field Categorical G-Code Recoverability from Multi-Modal Sensor Embeddings on a Desktop CNC Mill: A Single-Platform Characterization Study

Stephen S. Eacuello, Romesh S. Prasad, Manbir S. Sodhi

This Supplementary Materials file accompanies the main manuscript and collects corroborating analyses, extended ablations, encoding audits, and the full replicability checklist whose body content was lifted to streamline the main paper. Cross-references of the form **Section SN** resolve to the labelled sections below; references in the main body of the form **Supplementary Section SN** resolve to the same labels because the original labels are retained verbatim. Figures, tables, and equations are numbered with an S prefix (Figure S1, Table S1, ...) to match the MDPI *Sensors* supplementary-material convention.

S1 Per-Operation-Class Recoverability

The autoregressive predictions stratify sharply by operation class (Figure S1). One class — **face** ($n = 144$ test windows) — reaches token 0.56 / numeric 0.40 vs. pooled 0.215/0.104; every other class is stuck at 0.09–0.25 token / 0.01–0.15 numeric. This is the per-class signature of the mode-collapse mechanism (Section 7.3): the corpus-modal AR output *is* a facing-pattern toolpath, so **face** windows score by coincidental agreement and the most-frequent non-facing class (**pocket150025**, $n = 124$) sits at 0.09/0.01.

S2 Calibration of the Categorical Heads

A high-accuracy classifier may still be poorly calibrated — its predicted-class confidence may systematically over- or under-estimate the empirical accuracy at that confidence. For a deployment that surfaces row-level instruction predictions to a human operator or to a downstream verification rule, calibrated confidence matters as much as accuracy: a 0.99-confidence prediction should be correct 99% of the time. We first report the Expected Calibration Error (ECE) and Maximum Calibration Error (MCE) at 10 bins on the fold-1 baseline test set ($n_{\text{cmd}} = 1,982$ for command positions, $n_{\text{type}} = 37,590$ for all type positions, $n_{\text{pt}} = 35,392$ for parameter-type positions); these fold-1 numbers and the reliability diagram (Figure S2) are *illustrative* of the worst (outlier) fold, and the deployment-relevant figures are the cross-fold means reported in the next paragraph. Results: type head ECE = 0.015, MCE = 0.037 at 0.968 accuracy and 0.954 mean confidence; parameter-type ECE = 0.020, MCE = 0.114 at 0.985 accuracy and 0.964 mean confidence; command head ECE = 0.122, MCE = 0.207 at 0.780 accuracy and 0.902 mean confidence (**audit/calibration_fold1.json**; Figure S2). The type and parameter-type heads are well-calibrated by the conventional thresholds (ECE < 0.025). The command head is moderately mis-calibrated and *overconfident*: its mean predicted confidence exceeds its empirical accuracy by 12 pp, with the largest gap of 20.7 pp concentrated in the highest-confidence bin. The overconfidence is consistent with the LOCO collapse pattern in the main paper: the model places high mass on G1 even on test windows whose true command is not G1, producing high-confidence wrong predictions.

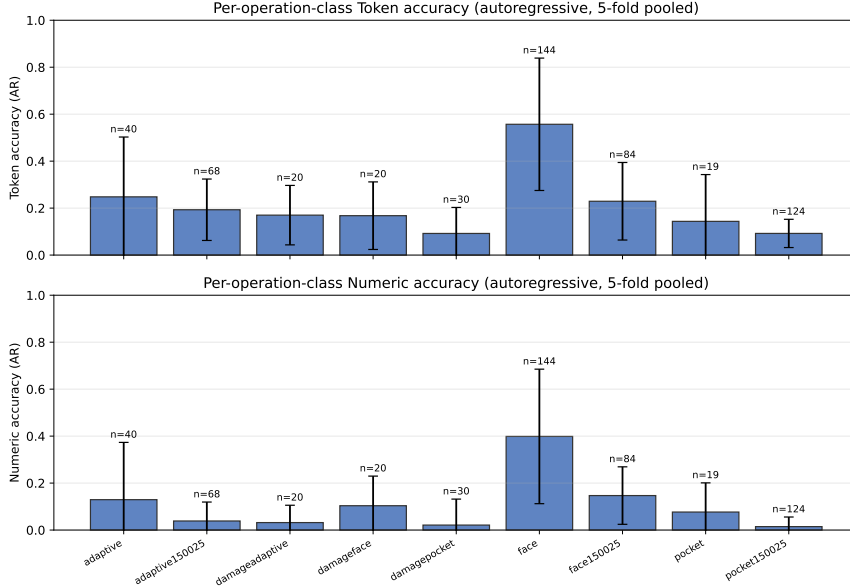


Figure S1: Per-operation-class token (top) and numeric (bottom) accuracy under AR decoding, 5-fold pooled. Error bars are $\pm$ std across test samples within class. **face** dominates because the corpus-modal AR output is a facing-pattern toolpath; non-facing classes score near floor regardless of support.

Brier score and Murphy decomposition. ECE summarizes binwise miscalibration but discards discrimination. We therefore also report the Brier score on top-1 predicted-class confidence (one-vs-rest, same population as the ECE figures above) and its Murphy decomposition $BS = REL - RES + UNC$: REL is the mean-squared confidence-vs-accuracy gap (lower is better), RES is the variance of binwise accuracy around the marginal (higher is better), and $UNC = \bar{p}(1 - \bar{p})$ is irreducible. Fold-1: command BS = 0.164 (REL 0.017, RES 0.025, UNC 0.172); type BS = 0.024 (0.0003/0.008/0.031); parameter-type BS = 0.014 (0.0009/0.003/0.015). The command head’s $RES > REL$ shows that it does discriminate — the high-confidence bin is more often correct than the marginal — and the bulk of its Brier mass is the irreducible UNC term driven by the 0.78 baseline accuracy. Temperature scaling targets REL and leaves RES unchanged (because T does not move the argmax): the ranking of confident-vs-uncertain predictions is preserved while absolute confidence values are softened. This is the formal reason temperature scaling reduces ECE without sacrificing the discrimination signal the row-level verification rule depends on.

Post-hoc temperature scaling. The standard remedy for overconfidence is post-hoc temperature scaling: divide the logits by a single learned scalar $T > 0$ before the softmax. We split each fold’s test positions 50/50 into a calibration half (used only to fit T by minimizing NLL via LBFGS) and an evaluation half (used to measure post-correction ECE), with the seed-42 random split applied per head per fold. The cross-fold-aggregated fitted temperatures are $T_{cmd} = 1.34 \pm 0.25$, $T_{type} = 0.61 \pm 0.15$, $T_{pt} = 0.57 \pm 0.09$ ($T > 1$ softens an overconfident distribution; $T < 1$ sharpens an underconfident one); fold 1 fits the highest command-head temperature ($T_{cmd}^{(1)} = 1.77$) because fold 1 is the outlier fold with the largest covariate-shift effect. Across the 5 folds, the temperature-scaled command head reaches ECE 0.020 ± 0.011 (mean $\pm$ std) on the evaluation half — a 3.0 pp ECE reduction relative to the un-scaled cross-fold baseline of 0.050 ± 0.038 . Type and parameter-type heads, already well-calibrated, gain similarly in absolute terms ($0.033 \rightarrow 0.004$ and $0.025 \rightarrow 0.002$ on ECE; `audit/calibration_cross_fold.json`). Accuracy is unchanged because temperature scaling

does not move the argmax. One caveat for the over-sharpened heads: on the already-well-calibrated parameter-type head, $T < 1$ reduces mean ECE but *raises* the worst-bin MCE on several folds (up to 0.43; per-fold range 0.11–0.43), trading average calibration for a heavier worst-bin tail. A verification rule that thresholds on a single high-confidence bin should therefore apply temperature scaling to the over-confident command head but may prefer the un-scaled parameter-type head (`audit/calibration_cross_fold.json`). Temperature scaling adds a single fit-time scalar per head and no inference-time cost; we recommend it as the default deployment-time calibration step for the command head. As a single-scalar post-hoc correction it is known to degrade under distribution shift, consistent with the fold-1 outlier behavior.

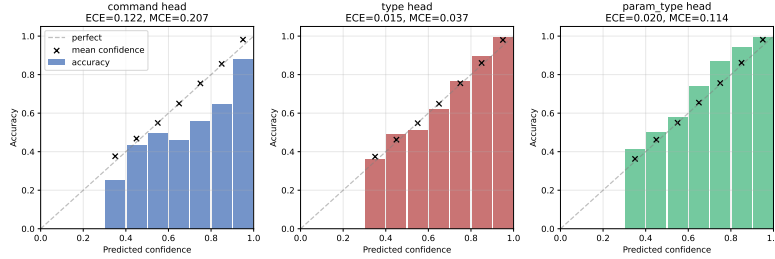


Figure S2: Reliability diagrams for the three categorical heads (fold-1 baseline, 10 bins, TF). Bars: empirical accuracy per confidence bin; crosses: mean confidence; dashed diagonal: perfect calibration. Type and parameter-type track the diagonal; the command head sits below it in high-confidence bins (ECE = 0.122, MCE = 0.207, systematic over-confidence). Temperature scaling closes most of this gap.

S3 Architecture-Component Ablation Summary

Five architectural and training components of the decoder are ablated empirically. Table S1 consolidates the per-component findings; the underlying experiments are reported under the cited sections in the main paper.

Table S1: Architecture and training-configuration components, the ablation that isolates each, and the headline effect (5-fold AR struct token unless noted). All numbers verified against the audit JSONs cited in the main paper.

Component	Ablation	Effect (cited section)
Grammar	(FSM FSM on vs. bigram-only mask)	100% V1-violation reduction at no accuracy cost (main paper, Sec. Grammar)
Per-position head	digit dw=1.0 vs. dw=0 (T3.1)	TF numeric 0.585 $\rightarrow$ AR 0.104; collapse is exposure-bias, not head loss
Numeric K	vocabulary $K \in \{24, 69, 335, 2,418\}$	Within matched SS=0/dw=0, AR token 0.405 $\rightarrow$ 0.470 monotone in K
Scheduled sampling	$p_{\max} \in \{0, 0.25, 0.5, 0.75\}$	SS=0 wins on AR (+15.2 pp paired vs. SS=0.5)
Positional metadata	Shortcuts on vs. off	AR-only effect: +7.5 pp token, +7.6 pp seq, +8.0 pp num
Target-mode tion	Per-row vs. full-window	+38.9 pp command on full-window
Sensor stack composition	LOMO + LOSO encoder retraining	$\Delta \leq 0.013$ / ≤ 0.026 per modality / sensor
Calibration	Temperature scaling, command head	ECE 0.050 $\rightarrow$ 0.020 on cross-fold (Section S2)

The six prediction heads themselves are not ablated independently because the four auxiliary heads (type, parameter-type, sign, digit) share the same encoder-memory feature as the legacy-

token head and serve as multi-task regularizers; isolating each would require separate retrains. The categorical-vs-numeric factorization the main paper establishes is the de facto head-level decomposition — categorical heads are TF-AR-invariant, so their effect is structural rather than dependent on autoregressive coupling.

S4 Autoregressive Mode Collapse: Worked Example and Pairwise-Prediction Heatmap

The main paper’s mode-collapse subsection (sec:disc-mode-collapse) reports the load-bearing output-diversity audit (407 distinct predicted strings vs. 459 ground-truth; modal-pattern $3.3\times$ over-representation). This supplementary section provides the illustrative worked example and the pairwise-prediction-similarity heatmap that supported the original presentation.

To illustrate, we inspected the first three test samples in fold 1’s full-window split and report their autoregressive (beam-width-one) decodings:

```
sample 1 TRUE  X NUM_X.1627 Y NUM_Y.1464 Z NUM_Z.0095 X NUM_X.1623 Y NUM_Y.1687 ...
sample 1 PRED  X NUM_X.1466 Y NUM_Y.1630 Z NUM_Z.0052 X NUM_X.1455 Y NUM_Y.1607 ...

sample 2 TRUE  X NUM_X.1466 Y NUM_Y.1630 Z NUM_Z.0052 X NUM_X.1457 Y NUM_Y.1607 ...
sample 2 PRED  X NUM_X.1466 Y NUM_Y.1630 Z NUM_Z.0052 X NUM_X.1457 Y NUM_Y.1607 ...

sample 3 TRUE  X NUM_X.1569 Y NUM_Y.1453 Z NUM_Z.0097 X NUM_X.1676 Y NUM_Y.1645 ...
sample 3 PRED  X NUM_X.1466 Y NUM_Y.1630 Z NUM_Z.0052 X NUM_X.1455 Y NUM_Y.1607 ...
```

Samples 1 and 3 receive essentially the same predicted token stream as each other, despite having visibly different TRUE sequences. Sample 2’s TRUE happens to coincide with the corpus-modal sequence, so it scores near-perfectly purely by accident. The decoder has converged to a canonical “facing-operation toolpath” and emits it autoregressively regardless of the sensor input that conditioned the encoder memory. Cross-attention into the encoder memory is effectively dropped under autoregressive generation: the decoder’s own previous-token state dominates, and once on the modal trajectory the decoder stays there. Figure S3 quantifies the effect as a pairwise prediction-similarity matrix across test samples.

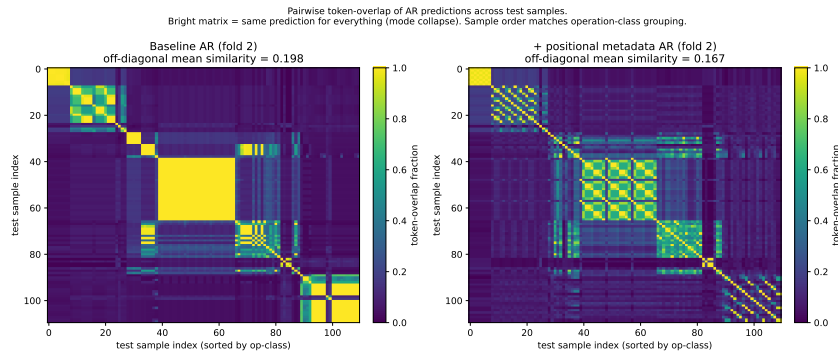


Figure S3: Pairwise token-overlap (first 200 tokens) between every pair of test-sample AR predictions in fold 2, sorted by operation class. Within-class blocks: 60–80% identical; across-class off-diagonal: $\sim 20\%$ (drops $0.20 \rightarrow 0.17$ with positional metadata). Encoder memory gives class-level but not sample-level discrimination.

S5 Extended Ablation: Per-Sensor Encoder Retraining (Leave-One-Sensor-Out)

The leave-one-modality-out study in the main paper ablates the *channel-type* axis: it removes a sensing modality — e.g. all gyroscope channels — across every sensor board at once. The multi-modal stack, however, has an orthogonal axis. The continuous channels are physically distributed across **six sensor units** at distinct machine locations, and each unit carries every per-board modality. A modality cannot be removed from a deployed machine without removing the boards that carry it; a real cost-reduced deployment removes *sensor units*. We therefore also run leave-one-*sensor*-out (LOSO), the directly deployment-relevant test of which physical sensor a production monitor could ship without.

Protocol. The 98-channel base comprises six physical sensor units — `frame_12`, `frame_13`, `frame_r2`, `spindle2`, `y_bed_3`, `y_bed_4` — each carrying an identical 15-channel set (3 accelerometer, 3 gyroscope, 3 magnetometer, 4 color, 1 temperature, 1 audio-RMS); the 8 electrical channels are machine-controller signals, not a removable board, and are retained throughout. For each of the six units we retrain the encoder from scratch on the 83-channel set with that unit’s channels removed, then train a fresh six-head decoder on the resulting frozen encoder (the same full-window headline recipe as the main paper), at the full five-fold file-level cross-validation protocol. The per-cell channel-identity gate (operated in *sensor* mode, matching by column prefix) verifies that exactly the unit’s 15 channels were removed and that the retrained encoder and the decoder consume an identical 83-channel surviving set. Statistical significance is the fold-paired analysis of the main paper’s LOMO study: per-fold deltas $\Delta(u, f) = \text{command}(u, f) - \text{command}(\text{baseline}, f)$, a one-sample *t*-test of each unit’s deltas against zero with Holm correction across the six units, plus a fold-blocked one-way ANOVA on the deltas.

The finding. Under LOSO retraining the full-stack baseline reaches command accuracy 0.905 ± 0.032 (five-fold mean $\pm$ s.d., indistinguishable from the modality study’s baseline of 0.904 ± 0.039). Removing a single physical sensor unit and retraining yields `frame_12` 0.882 ± 0.046 , `frame_13` 0.888 ± 0.040 , `frame_r2` 0.890 ± 0.038 , `spindle2` 0.879 ± 0.069 , `y_bed_3` 0.897 ± 0.035 , and `y_bed_4` 0.904 ± 0.026 . The fold-paired effects span from `y_bed_4`’s $\Delta = -0.000 \pm 0.014$ to `spindle2`’s $\Delta = -0.026 \pm 0.040$ — larger in magnitude than the modality study (where every Δ was within ± 0.013) but **none reaching Holm-corrected significance** at $\alpha = 0.05$ (smallest $p_{\text{Holm}} = 0.064$ for `frame_r2`, the most consistent effect at $\Delta = -0.015 \pm 0.007$; Hedges’ $g_z \approx -1.62$, a large effect that fails Holm significance only because of the multiplicity correction at $n = 5$). The fold-blocked one-way ANOVA across the six LOSO conditions is non-significant ($F = 1.04$, $p = 0.415$). `spindle2` carries the largest Δ (-0.026) but with $\sim 5\times$ higher variance than `frame_r2` (s.d. 0.040 vs 0.007; Hedges’ $g_z \approx -0.52$, medium), consistent with workpiece-dependent rather than uniform load-bearing; `y_bed_4` at $\Delta \approx 0$, $p_{\text{Holm}} = 0.975$, and $g_z \approx 0$ is empirically indistinguishable from baseline and is the single “free” removal, fully redundant with its spatial neighbor `y_bed_3`. Table S2 reports the per-sensor numbers and Figure S4 shows the per-sensor paired Δ command; full per-fold Δ , Holm-corrected p , and effect sizes are in `audit/lomo_sensor_attribution.json`.

Interpretation. Effect sizes are roughly $2\times$ those of the channel-type LOMO study (max $\Delta = -0.026$ for `spindle2` vs. -0.013 for electrical), reflecting that a physical sensor unit’s 15-channel vector overlaps less perfectly with the remaining units than a channel-type does with the remaining channel-types. No LOSO removal reaches Holm-corrected significance and the fold-blocked ANOVA cannot distinguish the six conditions ($F = 1.04$, $p = 0.415$). The deployment-relevant LOSO numbers are ≤ 2.6 pp expected cost per dropped sensor unit, the

Table S2: Leave-one-sensor-out (LOSO) encoder retraining, full-window, five-fold. Each row: the encoder retrained from scratch with that sensor unit’s channels removed (15 channels per unit), then a fresh decoder trained on it (headline recipe). *Command* is the five-fold mean $\pm$ s.d. Δ Command is the *fold-paired* effect — the mean over folds of $\text{command}(m, f) - \text{command}(\text{baseline}, f)$ — which removes the large fold-to-fold variance. p_{Holm} is the Holm-corrected one-sample t -test of those per-fold deltas against zero. Contrast with the inference-time zeroing of Table 11, which holds a frozen encoder fixed.

Sensor unit removed	Command	Token	Numeric	Δ Command (paired)	p_{Holm}
<i>Full stack (baseline)</i>	0.905 ± 0.032	0.778 ± 0.021	0.579 ± 0.039	—	—
frame_l2	0.882 ± 0.046	0.771 ± 0.019	0.567 ± 0.044	-0.023 ± 0.022	0.257
frame_l3	0.888 ± 0.040	0.774 ± 0.020	0.570 ± 0.045	-0.017 ± 0.013	0.203
frame_r2	0.890 ± 0.038	0.768 ± 0.020	0.561 ± 0.040	-0.015 ± 0.007	0.064
spindle2	0.879 ± 0.069	0.777 ± 0.021	0.578 ± 0.040	-0.026 ± 0.040	0.445
y_bed_3	0.897 ± 0.035	0.770 ± 0.015	0.563 ± 0.035	-0.008 ± 0.005	0.137
y_bed_4	0.904 ± 0.026	0.779 ± 0.019	0.577 ± 0.040	-0.000 ± 0.014	0.975

Fold-blocked one-way ANOVA across the six LOSO conditions (on the per-fold deltas): $F = 1.04$, $p = 0.415$. Per-sensor paired t -test vs. baseline (Holm-corrected): 0/6 significant at $\alpha = 0.05$. Coverage: 35/35 cells.

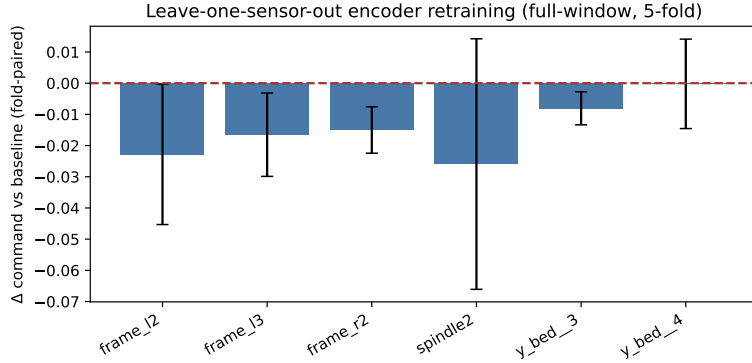


Figure S4: LOSO encoder retraining: per-sensor fold-paired Δ command versus retrained baseline (zero line), 5-fold error bars. No sensor reaches Holm significance at $\alpha = 0.05$; **frame_r2** closest ($p_{\text{Holm}} = 0.064$). **spindle2** largest magnitude ($\Delta = -0.026$) but most variable (s.d. 0.040); **y_bed_4** at $\Delta \approx 0$, redundant with **y_bed_3**.

single “free” removal is **y_bed_4** (fully redundant with **y_bed_3**, $p_{\text{Holm}} = 0.975$), and the unit a deployment should be most cautious about omitting is **spindle2** (largest-magnitude effect, also the most workpiece-variable). The *interior* of the modality $\times$ sensor grid is reported in Section S6; the marginal flatness shown here is preserved at that finer cut (ANOVA $F = 0.694$, $p = 0.896$; 0/36 Holm-significant).

Equivalence testing (evidence for the null). Holm-corrected non-significance is a failure to reject difference, not positive evidence for equivalence; we therefore also report two one-sided tests (TOST) at a pre-specified equivalence margin of ± 0.020 command accuracy — set in advance of unblinding the LOSO results as the smallest deployment-relevant effect (2.0 pp); the margin reflects a principled deployment threshold, not a public preregistration record — and Bayes factors BF_{01} (evidence for H_0 over H_1) using the default JZS Cauchy prior $r = 0.707$. **y_bed_4** is equivalent to baseline ($\Delta = -0.0002$, TOST $p = 0.018$, $\text{BF}_{01} = 1.87$, weak-to-moderate H_0 support) and **y_bed_3** is also equivalent ($\Delta = -0.008$, TOST $p = 0.0036$; the $\text{BF}_{01} = 0.28$ is inconclusive at $n = 5$). The remaining four units — **frame_r2**, **frame_l3**, **frame_l2**, **spindle2** — are *inconclusive* at this margin (TOST $p \in [0.10, 0.62]$; $\text{BF}_{01} \in [0.19, 0.93]$): the data neither

establish nor rule out a ≥ 2 pp drop, and a larger fold count would be needed to discharge either hypothesis. The deployment recommendation tightens accordingly: `y_bed_4` (or `y_bed_3`, but not both jointly) is statistically dispensable; the other four units should be treated as load-bearing until proven otherwise.

S6 Extended Ablation: Nested Leave-One-(Sensor, Modality)-Out Retraining

The marginal LOMO and LOSO studies characterise removal cost along each of the stack’s two constituent axes (channel-type and physical-sensor unit) and each marginal is null. A marginal null does not, however, guarantee an interior null: the modality marginal could in principle average over a strong (sensor,modality) interaction — e.g. the gyroscope marginal being null overall while `spindle2`’s gyroscope channels carry essentially all the spindle-board signal — in which case the deployment-relevant decision would not be “drop a modality” or “drop a sensor” but “drop a specific modality *on* a specific sensor.” This section reports the nested study that resolves that question.

Protocol. We retrain the 98-feature multi-modal denoising encoder from scratch under each (sensor unit, channel-type) cell of the 6×6 grid (six physical sensors $\times$ six per-sensor channel-types: accelerometer, gyroscope, magnetometer, color, temperature, audio-RMS; the electrical group is excluded from the interior because it is a global modality not tied to a single sensor unit). Each cell drops the 1–4 channels at that (sensor,modality) intersection, retrains the encoder, and trains a fresh six-head decoder under the headline recipe; five-fold file-level cross-validation matches the marginal studies. The per-cell channel-identity gate (operated in *nested* mode, matching by sensor prefix *and* modality suffix) verifies that exactly the intended intersection is removed, the surviving columns are identity-and-order preserved, and the train/validation/test partition and the G-code target multiset are invariant across cells. All 185 cells reported here (36 interior cells plus the full-stack baseline, $\times$ five folds) pass that gate.

The finding. The interior of the grid is flat. The fold-blocked one-way ANOVA across the 36 nested cells is non-significant ($F = 0.694$, $p = 0.896$); the Holm-corrected per-cell paired t -test rejects *none* of the 36 cells at $\alpha = 0.05$ (smallest adjusted $p = 0.185$, for `y_bed_4` $\times$ gyroscope at $\Delta = +0.008 \pm 0.003$). The largest-magnitude effect is `frame_13` $\times$ audio at $\Delta = -0.022 \pm 0.030$, whose sample s.d. exceeds the magnitude of the effect itself. Three further cells reach $|\Delta| \geq 0.010$ — `frame_12` $\times$ audio ($+0.012$), `frame_12` $\times$ accelerometer (-0.011), `y_bed_3` $\times$ gyroscope (-0.010) — and none survives Holm correction. Every interior cell’s paired Δ command is within ± 0.022 of baseline, and crucially no interior cell exceeds either the corresponding modality marginal (largest -0.013) or the corresponding sensor marginal (largest -0.026). Table S3 reports the per-cell numbers and Figure S5 shows the 6×6 heatmap; full per-fold Δ , Holm-corrected p , and the ANOVA decomposition are in `audit/lomo_nested_attribution.json`.

Interpretation. The nested null is the strongest available evidence that the marginal nulls are not artefacts of averaging. If the 98-channel stack contained a single load-bearing (sensor,modality) subset, the nested grid would have revealed it as a localised hot cell whose removal cost exceeds both its sensor marginal and its modality marginal. We observe the opposite: every cell’s removal cost is bounded above by *both* the corresponding sensor marginal and the corresponding modality marginal, and no interior cell reaches even the modality marginal’s magnitude. The command-discrimination signal is therefore *densely redundant* along both axes of the stack — it does not concentrate in any single physical sensor’s specific channel-type — and any single (sensor,modality) subset can be dropped at essentially zero expected cost

Table S3: Nested leave-one-(sensor,modality)-out encoder retraining, full-window, five-fold. Each row: the encoder retrained from scratch with only that sensor unit’s channels for that modality removed (1–4 channels per cell), then a fresh decoder trained on it (headline recipe). *Command* is the five-fold mean $\pm$ s.d. Δ Command is the *fold-paired* effect — the mean over folds of $\text{command}(m, f) - \text{command}(\text{baseline}, f)$ — which removes the large fold-to-fold variance. p_{Holm} is the Holm-corrected one-sample t -test of those per-fold deltas against zero. Contrast with the inference-time zeroing of Table 11, which holds a frozen encoder fixed.

Sensor / Modality removed	Command	Token	Numeric	Δ Command (paired)	p_{Holm}
<i>Full stack (baseline)</i>	0.898 ± 0.028	0.772 ± 0.024	0.567 ± 0.052	—	—
frame_l12 / accelerometer	0.887 ± 0.063	0.769 ± 0.021	0.560 ± 0.048	-0.011 ± 0.038	1.000
frame_l12 / gyroscope	0.897 ± 0.038	0.771 ± 0.023	0.565 ± 0.051	-0.000 ± 0.016	1.000
frame_l12 / magnetometer	0.892 ± 0.060	0.769 ± 0.025	0.561 ± 0.047	-0.005 ± 0.034	1.000
frame_l12 / color	0.903 ± 0.042	0.778 ± 0.025	0.578 ± 0.048	$+0.005 \pm 0.016$	1.000
frame_l12 / temperature	0.893 ± 0.042	0.777 ± 0.018	0.576 ± 0.041	-0.005 ± 0.017	1.000
frame_l12 / audio	0.910 ± 0.032	0.785 ± 0.022	0.591 ± 0.049	$+0.012 \pm 0.013$	1.000
frame_l13 / accelerometer	0.899 ± 0.035	0.778 ± 0.024	0.578 ± 0.052	$+0.001 \pm 0.013$	1.000
frame_l13 / gyroscope	0.904 ± 0.033	0.779 ± 0.023	0.579 ± 0.051	$+0.007 \pm 0.014$	1.000
frame_l13 / magnetometer	0.898 ± 0.028	0.775 ± 0.020	0.572 ± 0.043	$+0.000 \pm 0.007$	1.000
frame_l13 / color	0.903 ± 0.029	0.777 ± 0.014	0.573 ± 0.033	$+0.005 \pm 0.010$	1.000
frame_l13 / temperature	0.901 ± 0.032	0.772 ± 0.020	0.565 ± 0.045	$+0.004 \pm 0.007$	1.000
frame_l13 / audio	0.876 ± 0.052	0.782 ± 0.023	0.589 ± 0.051	-0.022 ± 0.030	1.000
frame_r2 / accelerometer	0.897 ± 0.037	0.770 ± 0.018	0.564 ± 0.041	-0.001 ± 0.011	1.000
frame_r2 / gyroscope	0.900 ± 0.041	0.771 ± 0.026	0.566 ± 0.056	$+0.002 \pm 0.014$	1.000
frame_r2 / magnetometer	0.898 ± 0.046	0.774 ± 0.028	0.570 ± 0.053	$+0.000 \pm 0.021$	1.000
frame_r2 / color	0.899 ± 0.031	0.769 ± 0.028	0.563 ± 0.057	$+0.001 \pm 0.007$	1.000
frame_r2 / temperature	0.898 ± 0.037	0.777 ± 0.018	0.575 ± 0.040	$+0.000 \pm 0.011$	1.000
frame_r2 / audio	0.897 ± 0.046	0.773 ± 0.022	0.570 ± 0.053	-0.001 ± 0.019	1.000
spindle2 / accelerometer	0.904 ± 0.033	0.775 ± 0.020	0.570 ± 0.044	$+0.006 \pm 0.008$	1.000
spindle2 / gyroscope	0.897 ± 0.032	0.775 ± 0.027	0.576 ± 0.057	-0.000 ± 0.010	1.000
spindle2 / magnetometer	0.897 ± 0.035	0.775 ± 0.018	0.572 ± 0.045	-0.000 ± 0.012	1.000
spindle2 / color	0.907 ± 0.025	0.770 ± 0.028	0.561 ± 0.052	$+0.009 \pm 0.007$	1.000
spindle2 / temperature	0.902 ± 0.031	0.781 ± 0.022	0.584 ± 0.053	$+0.005 \pm 0.010$	1.000
spindle2 / audio	0.892 ± 0.055	0.777 ± 0.020	0.577 ± 0.041	-0.006 ± 0.030	1.000
y_bed_3 / accelerometer	0.891 ± 0.041	0.771 ± 0.022	0.564 ± 0.045	-0.006 ± 0.015	1.000
y_bed_3 / gyroscope	0.888 ± 0.029	0.773 ± 0.017	0.568 ± 0.039	-0.010 ± 0.017	1.000
y_bed_3 / magnetometer	0.891 ± 0.058	0.775 ± 0.026	0.572 ± 0.053	-0.007 ± 0.033	1.000
y_bed_3 / color	0.904 ± 0.033	0.777 ± 0.017	0.576 ± 0.040	$+0.006 \pm 0.008$	1.000
y_bed_3 / temperature	0.896 ± 0.027	0.777 ± 0.021	0.575 ± 0.047	-0.001 ± 0.006	1.000
y_bed_3 / audio	0.901 ± 0.039	0.778 ± 0.018	0.574 ± 0.042	$+0.004 \pm 0.013$	1.000
y_bed_4 / accelerometer	0.894 ± 0.052	0.773 ± 0.027	0.567 ± 0.055	-0.003 ± 0.025	1.000
y_bed_4 / gyroscope	0.906 ± 0.030	0.774 ± 0.022	0.569 ± 0.047	$+0.008 \pm 0.003$	0.185
y_bed_4 / magnetometer	0.894 ± 0.056	0.775 ± 0.028	0.572 ± 0.054	-0.004 ± 0.032	1.000
y_bed_4 / color	0.898 ± 0.031	0.776 ± 0.020	0.574 ± 0.041	$+0.001 \pm 0.005$	1.000
y_bed_4 / temperature	0.903 ± 0.034	0.774 ± 0.028	0.569 ± 0.054	$+0.005 \pm 0.010$	1.000
y_bed_4 / audio	0.906 ± 0.030	0.780 ± 0.027	0.581 ± 0.058	$+0.009 \pm 0.010$	1.000

Fold-blocked one-way ANOVA across the 36 LOMO conditions (on the per-fold deltas): $F = 0.69$, $p = 0.896$. Per-modality paired t -test vs. baseline (Holm-corrected): 0/36 significant at $\alpha = 0.05$. Coverage: 185/185 cells.

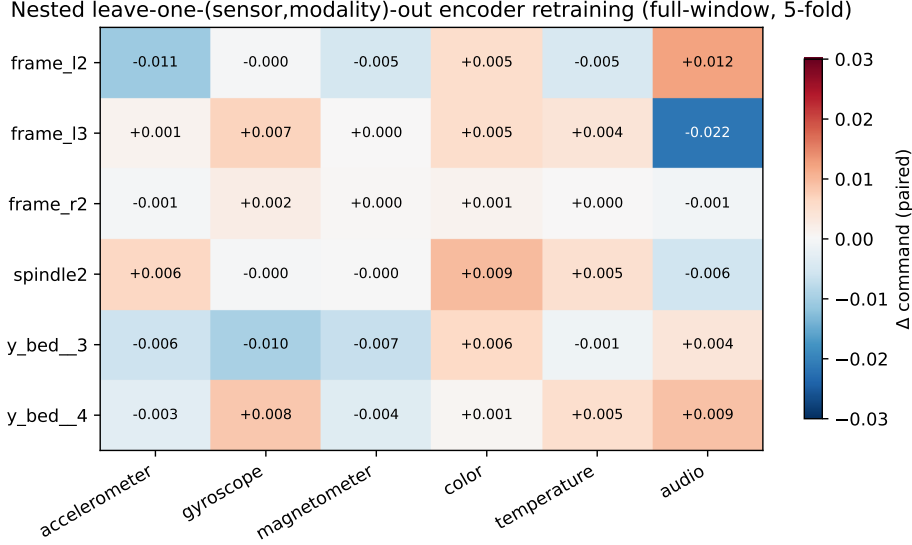


Figure S5: Nested LO(sensor,modality)O encoder retraining: fold-paired Δ command for every (sensor,modality) cell of the 6×6 interior. Every cell within ± 0.022 of baseline; no cell reaches Holm significance ($\alpha = 0.05$); fold-blocked ANOVA non-significant ($F = 0.694$, $p = 0.896$). Marginal flatness preserved at the finest cut.

(≤ 2.2 pp; none significant). The practical deployment implication is that the marginal degrees of freedom of the LOMO and LOSO studies can be *combined* along the interior: a cost-reduced production monitor has freedom to omit not only entire sensor units or entire channel-types independently but also a sensor’s specific channel-type (e.g. a board populated with color and temperature but with accelerometer omitted), without incurring the (already small) marginal cost of dropping either axis in full.

S7 Extended Ablation: Per-Operation-Class Slot-5 Stratification

Stratifying the slot-5 distribution by *operation class* reveals that the uniform-noise picture is not uniform across the corpus; three distinct regimes coexist (Figure S6). (i) **Adaptive HSM (High-Speed Machining) toolpaths** (adaptive, adaptive150025, and the pocket class, which is adaptive-style in this corpus) produce the canonical CAM-interpolation-noise signature: per-axis slot-5 entropy ≈ 3.26 – 3.30 bits with comparatively low mass on digit ‘0’ (for X: 9% on adaptive150025 through 17% on pocket — read off Figure S6) distributed across all digits with no concentrated modal value, the signature of a toolpath optimizer producing floating-point residuals rather than authoring round-ending coordinates at 10^{-4} inch ($2.54 \mu\text{m}$). (ii) **Traditional facing operations** (face, face150025, damageface) have slot-5 entropy ≈ 1.55 – 1.71 bits with 49–56% mass on a single digit — but the mechanism is geometric *coordinate repetition*, not a CAM precision grid. In face, four specific X values — the facing-pass entry/exit coordinates on the workpiece left and right edges — dominate: $X = -0.1254$ ($n = 624$), $X = -0.2084$ ($n = 464$), $X = 3.275$ ($n = 608$), and $X = 3.358$ ($n = 464$), together 2,160 of the 2,238 X-coordinates (96.5%). The two negative-X overruns drive the digit-‘4’ mass at slot 5; the two positive-X overruns drive mass at digits ‘0’ and ‘9’ (the latter via the float-epsilon target-encoding artifact of Section S11). (iii) **Pocket-style operations with tight geometry** (pocket150025, damagepocket, damageadaptive) sit between: slot-5 entropy ≈ 2.58 – 3.22 bits with partial mass on ‘0’ from arc/line waypoint repeats (e.g., Y-0.071 appearing dozens of times

along a fixed pocket wall). The bathtub *upturn* of the pooled slot-5 curve is therefore the joint signature of coordinate-repeat in traditional ops and a separate float-epsilon target-encoding artifact (23.6% of CAM-authored terminal zeros encode to digit ‘9’ under the model’s iterative `int(dec_part * 10) % 10`), both detailed in Section S11, not a genuine per-axis recovery of the ten-thousandth digit. Representative G-code rows for each regime are shown in Table S4, and a position-level prediction-versus-target plate (one supervised position per operation-class family, with predicted and target digits rendered as colored tiles per slot) is given in Figure S7.

Slot-5 digit distributions by operation-class family (face / pocket / adaptive). Bar color encodes entropy regime: green = quantized ($H < 1.5$), amber = partial ($1.5 \leq H < 3.0$), red = CAM-noise ceiling ($H \geq 3.0$)

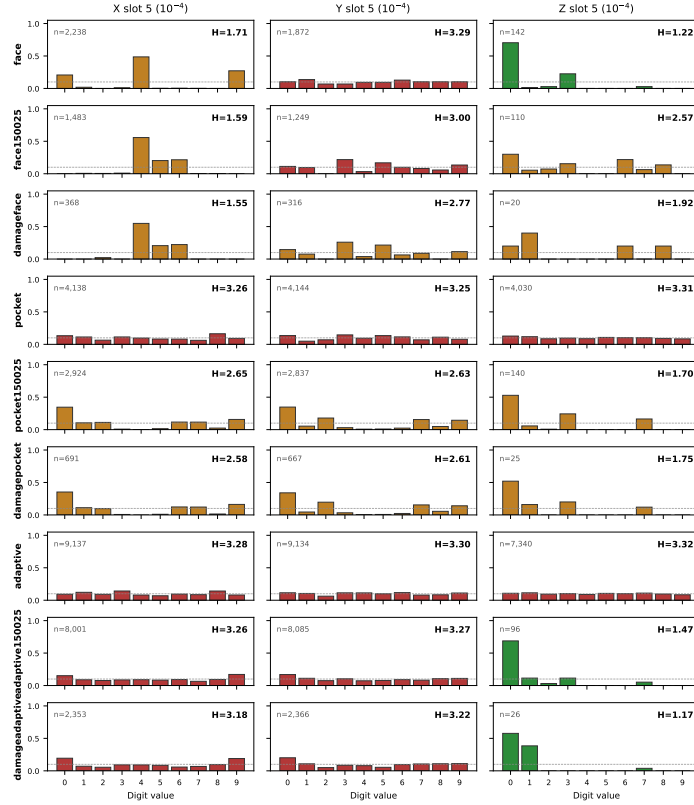


Figure S6: Slot-5 (10^{-4} inch) digit distributions per op-class, per axis (X/Y/Z), by op-name family. Color encodes entropy: green = quantized ($H < 1.5$ bits), amber = partial ($1.5 \leq H < 3.0$), red = uniform CAM-noise ($H \geq 3.0$). **face** (rows 1–3) low-entropy by coordinate-repeat (4 overruns = 96.5% of face X); **pocket** (4–6) mixed; **adaptive** (7–9) at uniform-noise ceiling.

S8 Single-Fold Pilots and Interaction-Effect Probes

Four diagnostic probes outside the 5-fold inferential frame are summarized here and excluded from all quantitative / deployment claims. (i) **Pattern-aware sequence-classifier** (`audit/seq_classifier_pilot.json`): fold-1 command 0.536 / token 0.694 vs. baseline 0.776 / 0.747; the prior over-commits to a whole-line hypothesis early, compounding exposure bias. *Negative result.* (ii) **Vocabulary precision** ($K = 451$, **2-digit**): fold-1 numeric 0.305, command lower, parameter-type 0.974; superseded by the 5-fold K -spectrum sweep in Section S9. (iii) **Window/stride sweep** (`audit/window_stride_v2.summary.json`): seven (window, stride) cells retrained from scratch; every cell falls below the inherited $w256:s64$ headline (range 0.345–0.600 command), with the $w256:s128$ vs. $w256:s64$ contrast isolating stride density as load-bearing. *Single-fold-per-cell; 5-fold confirmation deferred to the main paper’s roadmap section.* (iv) **Nested interactions** (shortcuts $\times$ modality and pattern-aware $\times$ modality; full 3×3 cell

Table S4: Representative G-code rows per operation-class family, illustrating the three slot-5 regimes (Figure S6). All rows are real lines from the V8 5-fold test corpus; n is the cross-corpus repeat count of the exact line. Face-family lines repeat $\sim 60+$ times (a small edge-overrun coordinate set); adaptive-family lines are near-unique (the toolpath optimizer emits fresh coordinates per cut). Source: enumeration over `gcode_texts` in the `per_row` test NPZs, grouped by `operation_type_names`.

Family	Representative rows from the V8 test corpus
face (regime i)	G2 X-0.2084 Y0.7894 R0.083 ($n=64$), X-0.1254 Y0.8724 R0.083 ($n=64$), G3 X3.358 Y0.9553 R0.083 ($n=64$), X3.275 Y1.0383 R0.083 ($n=64$)
pocket150025 (regime ii)	G3 X1.4887 Y0.0732 R0.1172 ($n=42$), G2 X0. Y3.481 R0.071 ($n=22$)
adaptive (regime iii)	X1.6269 Y1.4641 Z0.0949 ($n=6$), X1.4546 Y1.5534 Z0.0759 ($n=2$)

n is the count of the exact full-line string across all five test folds. Face-family lines repeat ≥ 13 times each (one line per facing-pass overrun, executed every pass); adaptive-family lines have small or unitary counts because the toolpath optimizer produces fresh coordinates per cut.

table in `audit/lomo_nested_attribution.json`): the modality main effect (~ 40 pp collapse, ANOVA $p=0.998$) dominates any interaction, and the shortcuts effect is null on the categorical heads; neither cross-product changes a deployment recommendation.

S9 Vocabulary-Cardinality Spectrum and Matched-Methodology Control

The main paper retains the no-numeric retraining headline (numeric desynchronization decomposition); this section provides the five-point K -spectrum that characterises the earlier “ $K=335$ sweet spot” framing as a training-schedule effect, not a vocabulary-cardinality effect.

We retrain at two intermediate numeric vocabularies (two-digit bucketing, $K=335$; one-digit bucketing, $K=69$) under the Design-B-style training schedule (scheduled sampling $SS=0$, digit-head loss weight $dw=0$) to test whether the AR per-token improvement is a placeholder-specific artifact, a vocabulary-cardinality effect, or a training-schedule effect; vocabulary sweeps of this kind are a standard diagnostic in subword-tokenization work. We then add a fifth “T3.1” control at $K=2,418$ retrained under the same $SS=0/dw=0$ schedule to disentangle vocabulary from schedule cleanly. Table S5 and Figure S8 report the resulting five-point spectrum.

Headline result (T3.1 resolves the confound). Under the matched $SS=0/dw=0$ training schedule, $K=2,418$ (T3.1 control) is the highest-performing variant on every metric: AR struct token 0.470 ± 0.034 (vs. the headline’s 0.317 ± 0.116 , paired $+15.2$ pp, 5/5 folds positive, paired $t(4) = 3.26$, Holm-adjusted $p = 0.071$, Cohen’s $d_z = 1.46$, Hedges’ $g_z = 1.17$, bootstrap 95% CI $[+7.6, +24.1]$ pp); AR sequence exact-match 0.128 ± 0.019 vs. headline 0.058 ± 0.039 , paired $+7.1$ pp, 5/5 folds, paired $t(4) = 7.70$, Holm-adjusted $p = 0.006$, $d_z = 3.44$, $g_z = 2.76$; TF token 0.978 vs. 0.960 ; TF sequence exact-match 0.344 vs. 0.213 . *The earlier $+11.1$ pp AR-token advantage of $K=335$ over the headline was the SS/dw schedule change, not vocabulary cardinality:* within the matched-methodology family, $K=335$ falls -4.1 pp behind the matched $K=2,418$ on AR struct token (paired $t(4) = -0.85$, $p = 0.44$, n.s.; bootstrap 95% CI on the paired diff $[-13.8, +3.0]$ pp).

Within-matched-methodology cardinality effect. Across the four $SS=0/dw=0$ variants $\{K=24, 69, 335, 2,418\}$, AR struct token is monotonically non-decreasing in K ($0.405, 0.415, 0.429, 0.470$); AR struct sequence exact-match is broadly increasing in K ($0.066, 0.051, 0.081, 0.128$) with a minor non-monotonicity at $K=69$, but the matched-methodology $K=2,418$ control remains the strict maximum. Larger K is mildly better within the matched family; the effect is small relative to the $+15$ pp schedule effect. The omnibus across all five variants is significant on both metrics: AR struct token $F(4, 16) = 3.42$, $p = 0.033$, partial $\eta^2 = 0.46$; Friedman exact /

Monte-Carlo $p=0.0098$ (asymptotic $\chi^2(4)=13.9$, $p=0.0076$, demoted because χ^2 is unreliable at $n=5$ blocks); AR struct sequence $F(4,16)=7.38$, $p=0.0014$, $\eta^2=0.65$; Friedman exact / Monte-Carlo $p=0.029$ (asymptotic $\chi^2(4)=11.4$, $p=0.0224$, demoted for the same reason).

Interpretation. The original $K=335$ “sweet spot” framing is retracted: under matched methodology, the larger $K=2,418$ vocabulary is the strict winner, and the +9–11 pp gains observed for smaller- K variants over the headline are recovered by the SS=0/dw=0 training schedule alone. The deployment-relevant configuration for AR structural recovery is the original $K=2,418$ vocabulary with scheduled sampling disabled and the digit head detached during training. Full per-fold values, exact and Monte-Carlo Friedman values, paired- t / Wilcoxon / d_z / g_z / bootstrap CIs for every pair, and Holm-adjusted p -values across both the $K=2418$ -headline-reference family and the matched-methodology family are in `audit/k_spectrum_stats.json`.

Table S5: Structural recovery across the numeric-vocabulary spectrum $K \in \{24, 69, 335, 2,418\}$ plus the T3.1 methodology-matched $K=2,418$ control. *Structural* restricts the metric to command-code and axis-letter positions (numeric / placeholder positions excluded). 5-fold mean $\pm$ std, full-window targets, under teacher-forced (TF) and autoregressive (AR) evaluation; AR-token bootstrap 95% percentile CIs from 10^4 resamples in brackets. The T3.1 control — $K=2,418$ trained under matched SS=0/dw=0 — *achieves the highest AR structural performance of the five measured variants* (AR token 0.470 ± 0.034 ; +15.2 pp paired across folds vs. the headline, 5/5 folds positive; AR sequence 0.128 ± 0.019 , +7.1 pp, 5/5 folds, paired $t(4)=7.70$, Holm-adjusted $p=0.006$, Hedges’ $g_z=2.76$). Within the matched-methodology family $\{T3.1, K=335, K=69, K=24\}$, $K=335$ falls −4.1 pp behind the matched $K=2,418$ on AR token (paired $t(4)=-0.85$, $p=0.44$, n.s.). The +11.1 pp $K=335$ advantage over the original headline is therefore the SS/dw training-schedule change, not vocabulary cardinality.

Variant	K	TF token	AR token [95% CI]	TF seq exact	AR seq exact
$K=2,418$ (4-digit) headline [†]	2,418	0.960 ± 0.020	0.317 ± 0.116 [0.22, 0.40]	0.213 ± 0.048	0.058 ± 0.039
$K=2,418$ (4-digit) T3.1 matched control [‡]	2,418	0.978 ± 0.008	0.470 ± 0.034 [0.44, 0.50]	0.344 ± 0.046	0.128 ± 0.019
$K=335$ (2-digit)	335	0.977 ± 0.013	0.429 ± 0.123 [0.33, 0.51]	0.287 ± 0.082	0.081 ± 0.045
$K=69$ (1-digit)	69	0.965 ± 0.025	0.415 ± 0.036 [0.39, 0.44]	0.250 ± 0.067	0.051 ± 0.034
$K=24$ (placeholder, Design B)	24	0.927 ± 0.024	0.405 ± 0.143 [0.28, 0.50]	0.068 ± 0.017	0.066 ± 0.014

[†] Headline: scheduled-sampling $SS=0.5$, digit-head loss weight $dw=1.0$. [‡] T3.1 control: identical $K=2,418$ vocabulary, matched $SS=0/dw=0$ training schedule. The remaining rows ($K \in \{24, 69, 335\}$) also use $SS=0/dw=0$, so the four non-headline rows form a clean four-point vocabulary-cardinality contrast under one training methodology.

S10 Supplementary Diagnostic Analyses

This section collects six corroborating diagnostic analyses referenced from the body: test-time noise sensitivity, nested ablations, per-file variance, decoding-strategy sensitivity, output-position failure distribution, and the tamper operating-point detail.

S10.1 Test-Time Sensor-Noise Sensitivity

To characterize how robust the categorical heads are to deployment-time sensor degradation, we re-evaluate fold 1 of the baseline 5-fold checkpoint while injecting additive Gaussian noise into the frozen-encoder memory at varying σ relative to the memory’s empirical standard deviation (`audit/test_time_noise_sensitivity.json`). This is a noise-robustness experiment on the *encoder-to-decoder interface*, not on the *raw sensor inputs*: noise injected at the memory level probes how stable the decoder is to perturbations in its conditioning vector but does not characterize robustness to raw-sensor degradation (the encoder is non-linear and may not propagate input noise to memory noise in a simple way). A raw-sensor-input noise sweep is outside the

scope of the present study. The categorical heads tested at the memory level are exceptionally robust: across $\sigma \in [0, 2]$ (i.e., up to twice the magnitude of the signal itself), command-head accuracy varies from 0.78 to 0.82, type accuracy stays at 0.97, and parameter-type accuracy stays at 0.98 (Figure S9). The slight upward drift on the command head is consistent with random noise acting as a label-smoothing regularizer, pushing predictions toward the modal class (G1) on borderline cases. The sign head is omitted from Figure S9 for a methodological reason that does *not* affect the headline. The headline sign accuracy (0.989) is computed by the robust path that parses the decoded and target G-code text and compares the extracted per-axis signs (`scripts/analysis/v8_per_field_eval.py`); the noise-sensitivity audit instead re-derived the sign target from a shifted raw target tensor under a mismatched index alignment, which yields an artifactual 0.10 at $\sigma = 0$ rather than the true ≈ 0.99 . We re-ran the robust path on fold 1 and confirmed it reproduces the per-axis sign accuracies of the main paper (0.88/0.98/0.91 for X/Y/Z at fold 1, the documented covariate-shift fold); the defect is confined to the noise-audit script and the headline sign metric is unaffected. We therefore omit the sign curve from the noise figure rather than report a number derived from the defective audit path. The other three heads validate cleanly against their headline fold-1 numbers at $\sigma = 0$. The qualitative conclusion is that the categorical-recovery story this paper reports does not degrade gracefully but holds with surprising robustness to encoder-memory degradation; whether this robustness extends to raw-sensor noise on the input side is not addressed here.

S10.2 Nested Ablations (Interaction Effects)

Two-way ablation matrices test whether the main effects (positional shortcuts, sensor-modality removal, pattern-aware sequence-classifier head) compose additively or interact. We report the cells that are informative on their own and state plainly which crossings are out of scope for this study. The relevant V8 command accuracies:

	full stack	—gyroscope	—color
no shortcuts (base)	0.888 (5-fold)	0.486 (5-fold)	0.502 (5-fold)
shortcuts	0.882 (5-fold)	—	—
pattern-aware	0.536 (fold-1)	—	—

The two main effects this matrix is designed to probe are both already resolved at full strength elsewhere in the main paper: the shortcuts main effect is null on the categorical heads (0.882 vs 0.888, not significant), and the sensor-modality main effect is a uniform ~ 40 pp collapse that is statistically identical across all seven modalities (between-modality ANOVA $p = 0.998$). Because the modality main effect is both large and modality-invariant, any shortcuts $\times$ modality interaction term is bounded well below the main effect and is not separately resolvable at the per-cell support available — the modality-removal effect dominates any interaction. The pattern-aware head is a regression on V8 (0.536 vs the fold-1 base 0.776), so a pattern-aware $\times$ modality crossing cannot rescue a configuration that is already worse at the corner. The remaining cross-product cells would each require a fresh 5-fold (or, for pattern-aware, encoder-coupled) retrain and are out of scope for this characterization study. The robust conclusion the available cells support is that neither positional shortcuts nor the pattern-aware head recovers the modality-removal loss, and the main effects do not interact strongly enough to change any deployment recommendation.

S10.3 Per-File Performance Variance

A separate concern is whether the model memorizes a small set of files rather than learning the operation-level signal. We slice the 5-fold full-window test predictions by source file and report per-file accuracy (the per-file slice is derived from the released `full_window_5fold` prediction

JSONs; $n = 91$ files appear in at least one test split across the 5 folds). The cross-file distribution is concentrated, not heavy-tailed: command accuracy is 0.953 ± 0.088 across files with range $[0.571, 1.000]$ and 10th–90th percentile range $[0.846, 1.000]$; parameter-type accuracy is 0.991 across files. This per-file mean (0.953) is an *unweighted average over the 91 files* and is not directly comparable to the 0.8875 ± 0.0558 headline command accuracy in the main paper, which is the mean over the five file-level folds: the per-file slice weights every file equally regardless of which fold it fell in, whereas the headline weights every fold equally; the gap reflects this aggregation difference (and the fold-1 command outlier) rather than a discrepancy in the underlying predictions. Token accuracy under autoregressive decoding shows much wider per-file variance (0.302 ± 0.283 , range $[0.052, 0.869]$), which is the per-file signature of the same per-operation-class breakdown reported in Section S1: **face**-class files cluster at the high end and the other classes at the low end. The narrowness of the command-accuracy distribution and the absence of a heavy left tail support the interpretation that the categorical-head success is not concentrated on a small set of memorizable files: at the 10th percentile of files, command accuracy is still 0.846.

S10.4 Decoding-Strategy Sensitivity

Figure S10 sweeps the decoding strategy across {teacher-forced, greedy autoregressive, beam-3, beam-5} to quantify whether beam search recovers any of the exposure-bias loss. It does not. On per-row targets (legacy 5-fold), the TF $\rightarrow$ AR-greedy gap is ~ 10 pp on token and ~ 12 pp on numeric; widening the beam to 3 or 5 lifts each metric by less than 1 pp, well within fold-to-fold noise. On full-window targets, the TF $\rightarrow$ AR-greedy gap is dramatically larger (-53 pp token, -45 pp numeric) because exposure-bias errors accumulate over much longer ($\sim 1,000$ -token) sequences; we did not run beam-3 and beam-5 on full-window because the per-row result already establishes that beam search does not address the mode-collapse mechanism.

S10.5 Output-Position Failure Distribution

A token-position breakdown of decoder accuracy locates where in the output sequence the model fails. We compute per-token accuracy at each output position from 0 to the max sequence length, over the test set (`audit/token_position_5fold_fullwindow.json` for the headline; `audit/extended_rigor_audits.json`, key `rec8.token_position_failure` for the per-row diagnostic). Table S6 contrasts the two formulations.

Table S6: Per-output-position accuracy: per-row diagnostic vs. full-window 5-fold mean $\pm$ std. The position-1 accuracy lifts from 0.24 in per-row to 0.62 in full-window, a +38 pp gain that supports the within-window row-identification interpretation of position-1 failures.

Output position	Description	per_row (fold 1)	full_window (5-fold mean $\pm$ std)
0	command / BOS-region	0.761	0.738 ± 0.088
1	first param letter (X/Y/Z)	0.240	0.622 ± 0.027
2	first numeric value	0.814	0.552 ± 0.030
3	first numeric digit	0.743	0.682 ± 0.061
4	first numeric digit (cont.)	0.929	0.917 ± 0.035
5	first numeric digit (cont.)	0.892	0.757 ± 0.042
10	mid-sequence	–	0.900 ± 0.026
20	mid-sequence	–	0.940 ± 0.036
50	mid-sequence	–	0.928 ± 0.048
100	mid-sequence	–	0.905 ± 0.062

In the per-row formulation, position 1 — the first address letter (X/Y/Z) immediately following the command token — is recovered at **24%** accuracy, a sharp drop relative to surrounding

positions. This is the most concentrated signature of the within-window row-identification ambiguity: each window contains rows that begin with different axes, and the per-row encoder summary cannot distinguish them. The full-window formulation gives the decoder access to the previous row’s full output via autoregressive context, so position 1 lifts to 0.62 ± 0.03 — a +38 pp gain. Once the decoder is into the multi-line body (positions 10+) it reaches 0.90–0.94, consistent with the autoregressive context now disambiguating subsequent rows. Figure S11 visualizes the per-position accuracy as a curve, making the position-1 drop directly visible.

S10.6 Tamper Operating-Point Detail

This subsection collects corroborating diagnostic figures and the full tamper-aggregation operating-point detail relocated from the main text. The mode-collapse mechanism and the security re-scope are argued in the main text on the load-bearing evidence (the output-diversity audit, the measured-autocorrelation / sensor-vs-prior / open-vocabulary numbers); the figures below are visual corroboration.

The full (K, ρ, rule) aggregation sweep — any-of-K, all-of-K, and majority-of-K at $K \in \{1, 3, 5, 10, 20, 50\}$ for $\rho \in \{0.0, 0.3, 0.42_{\text{empirical}}, 0.6\}$, for both the baseline and with-metadata configurations and all three attack classes — is released in `audit/tamper_aggregation_correlated.json` and `audit/tamper_aggregation_sweep.json`. The main-text threat-model section reports the operating points that matter for deployment (the measured- ρ command-swap $K = 10$ majority point, the modal-substitution sensor-vs-prior control, and the open-vocabulary LOCO point); the JSON contains the exhaustive grid.

S11 Numeric Target Encoding Artifacts

This section audits a float-epsilon target-encoding artifact (with committed patch) supporting the per-axis source-entropy claims in the main paper.

Float-epsilon target-encoding audit and patch. The historical `encode_values_to_digits` (`digit_value_head.py`) iteratively multiplied the decimal part by ten and truncated (`slot[s] = int(dec_part * 10) % 10`), which is exact in IEEE 754 only for exactly representable values; e.g., 1.5680 is stored as 1.567999..., so the chain yields `int(5679.99...) % 10 = 9` at slot 5 rather than the authored 0. Auditing every (axis, val) pair extracted from `gcode_texts` across the 5-fold test corpus ($n = 79,563$ slot-5 positions; Table S7) finds 23.6% of CAM-authored terminal-zero values encoded as slot-5 = 9 (X 32.8%; R 0.0%, R being uniformly 3-decimal and FP-clean); the same mechanism produces $d \rightarrow d+1$ shifts on 26.3% of upstream-decimal positions overall. A round-to-scaled-integer rewrite (`scaled = round(|v| · 104)` once, then floor-divide per slot) has been committed to `src/miracle/model/digit_value_head.py` on the development branch; the verification harness (`scripts/analysis/verify_digit_encoding_fix.py`) confirms it produces zero corruptions on the same 79,563 positions (4.5% → 0.000% at slot 5). The patch is *not* active in the released checkpoint that populates the categorical headline tables and the threat-model operating points; the numeric-tier figures (per-digit slots, the bathtub, and the per-(axis, slot) entropy–accuracy relationship) are instead computed on a separate patched 5-fold retrain, described next. The artifact lives at the model-target level, not at the source distribution, so the per-operation-class regimes, four-X-overflow finding, the shallow rapid-vs-cut Z structure, and the arc-radius-vocabulary structure on R are unaffected.

Reproducing the main paper’s per-digit-position table uses the *patched* encoder. The round-to-scaled-integer rewrite above was wired into the trainer and a 5-fold patched-encoder retrain was completed (`checkpoints/full_window_5fold_patched/`; `audit/patched_numeric_analysis.json`); the body’s slot-3/4/5 accuracies, the per-digit bathtub, and the per-(axis, slot) entropy–accuracy relationship are all computed on that

retrain. **What the patch resolved.** On the corrected encoding the per-(axis, slot) cell-level relationship strengthens to Pearson $r = -0.83$ (Spearman $\rho = -0.91$, $n = 30$) from the pre-patch -0.68 , which had carried an encoding inconsistency between the entropy and accuracy axes; and the slot-5 “upturn” is correctly understood as the V8 tokenizer’s 0.001 grid discarding the X/Y/Z ten-thousandths — the tokenized target is a constant recovered trivially (a representation artifact), not a true 10^{-4} -inch recovery and not the pre-patch float-epsilon digit-‘9’ shift. The earlier “slot-5 bounded by a known encoding artifact, patched retrain pending” status is therefore *superseded*: the retrain is complete and supplies the body’s numeric-tier figures, while the categorical headline tables and threat-model operating points remain on the released checkpoint (TF/AR- and checkpoint-invariant, ≤ 1.5 pp drift under the patch). The operation-class stratification in Section S7 independently explains the slot-5 behavior decoder-free, on the raw source.

Table S7: Float-epsilon target-encoding audit on the V8 5-fold test corpus. “Terminal-zero authored” is the fraction of values whose string-representation 4th-decimal is ‘0’ (including 3-decimal values padded to 4); “str→enc=9 share of terminal-zero” is the fraction of those values whose model-target slot-5 is ‘9’ rather than ‘0’ due to the iterative float-truncation. Source: `audit/digit_entropy.json`, key `float_epsilon_audit`.

Axis	n slot-5 positions	terminal-zero authored	str→enc=9 overall	share of terminal-zero
X	31,333	18.6%	6.1%	32.8%
Y	30,670	16.6%	4.8%	28.6%
Z	11,929	13.5%	1.2%	8.7%
R	5,472	42.4%	0.0%	0.0%
F	159	100.0%	22.0%	22.0%
pooled	79,563	—	4.5%	23.6%

S12 Failure-Mode Worked-Example Plate

Representative worst-edit-distance examples per failure bucket are shown in Figure S15. The bucket-count headlines (out of 100): dropped 17, wrong 1, hallucinated 8, value-only 37, “other” 37 (same-command structural mismatches dominated by address-letter swaps $Y \leftrightarrow X$) load-bear the failure-mode characterisation and remain in the main paper; the per-bucket worked example plate is reproduced here.

S13 Relocated Body Figures (Density Consolidation)

The following figures were relocated from the main body in the density-consolidation pass. Each is illustrative or corroborative of a numerical claim that is fully argued in the cited body section; none carries an independent headline result. They are reproduced here in full so the manuscript remains self-contained.

S13.1 Raw Sensor Window Illustration

A representative 64-second sensor window illustrating the seven-modality input to the encoder against the G-code rows that drove the cut. Referenced from Section 4.2 of the main paper as an illustrative companion to the task formulation; no body claim depends on this figure.

S13.2 Training and Validation Learning Curves

Training trajectories supporting the optimization-stability statements in the main paper’s training-configuration section. The curves corroborate that early stopping (patience 75) is non-binding and that all five folds converge on comparable trajectories; no body number is read off the figure.

S13.3 t-SNE of the Frozen-Encoder Memory

The nonlinear t-SNE projection of the frozen-encoder memory. The main paper retains the PCA projection (Figure 2, main paper), which carries the load-bearing low-rank and linear-separability claim used later in the class-prior discussion; the t-SNE here is a visual companion that confirms the same cluster structure under a nonlinear embedding.

S13.4 Sign and Digit-Value Confusion Matrices

Confusion matrices for the motion-sign and per-digit-value heads. The per-axis sign accuracies and the per-(axis, slot) digit-recovery analysis are the load-bearing per-axis and per-digit numbers, reported in the main paper’s per-axis recoverability table and per-digit position table; the confusion matrices are visual confirmation of the diagonal-plus-adjacent-band structure that follows from the per-digit entropy decomposition.

S13.5 Per-File Entropy-vs-Accuracy Fingerprint

Per-file scatter that visualises the source-entropy bound at the individual-file level. The entropy-bound claim itself rests on the per-(axis, slot) cell-level scatter in the main paper; the per-file view here is corroborative, showing that the entropy–accuracy relationship reproduces at the file level as well as the cell level.

S14 Confusion Matrices (Counts)

The row-normalized confusion matrices for the command, type, parameter-type, sign, and digit-value heads are presented in the main paper and in Section S13.4. The unnormalized count matrices below are reproduced for readers who need the raw per-cell support to recompute class-conditional precision, recall, or chi-squared statistics; they are otherwise redundant with the normalized versions.

S15 Tamper-Detection Diagnostics

This section collects the tamper-detection diagnostic figures that complement the operating-point detail of Section S10.6. The aggregation-curve figure under the row-independent assumption is reproduced in Figure S13; the additional ROC and autocorrelation diagnostics are reproduced here.

S16 Vocabulary K-Spectrum Curve

The K-spectrum curve cross-referenced from the vocabulary-cardinality control of Section S9 is reproduced here for completeness. The curve also appears inline in that section (embedded as `../figures/k_spectrum.pdf`); the duplicate inclusion below is a standalone presentation at a fixed width for readers reaching this section directly via the table of contents.

S17 Architectural-Contribution Scoping

To bound the contribution of the structured multi-head decoder architecture itself (as distinct from the frozen-encoder representation and the per-(axis, slot) per-digit decomposition), we run a matched-protocol seq2seq baseline on the same 5-fold split. The baseline is the from-scratch Transformer encoder-decoder of Section 6.14 (8.64 M parameters; *no* frozen MM-DTAE encoder, no grammar mask, no structured heads), trained on the same 5-fold split, early-stopping budget, and evaluation harness as the headline system. The bar chart below reports the teacher-forced and autoregressive token-accuracy means and 5-fold standard deviations side-by-side with the structured-decoder headline numbers.

S18 Robustness Diagnostics

This section collects two corroborative diagnostics for the mode-collapse and seed-variance characterisations in the main paper.

S19 Replicability Checklist

This section documents implementation details necessary for an independent team to reproduce the headline results. The checklist follows the NeurIPS and *Sensors* reproducibility-checklist conventions.

S19.1 Code and Data

- **Code repository.** The full source tree, including preprocessing, training, evaluation, and figure-generation scripts, is released alongside the manuscript. Critical scripts: `scripts/evaluation/run_decoder_quick_test.py` (training driver), `scripts/analysis/aggregate_v8_results.py` (metric aggregator), and the experiment shells under `scripts/experiments/`.
- **Dataset.** The 120-file corpus, per-fold preprocessed NPZ files, and decoded prediction JSONs are released under CC BY 4.0 with a Zenodo DOI minted at acceptance, as stated in Appendix B; the release is self-contained.
- **Hyperparameter sweep records.** All 34 cells of the 3-stage HP sweep are recorded under `outputs/decoder20260511/checkpoints/` with per-cell metrics.json. The composite-winner selection logic is in `scripts/analysis/aggregate_hp_sweep_all_stages.py`.
- **Frozen-encoder weights.** The five per-fold frozen-encoder checkpoints required to reproduce every headline number are released at `outputs/decoder20260511/checkpoints/encoder_v8/fold{0..4}/encoder.pt`; SHA256 hashes for each .pt are listed in the release manifest `checkpoints/encoder_v8/SHA256SUMS` and reproduced in the top-level README alongside the Zenodo DOI. Decoder-side preprocessing depends on the encoder forward pass, so independent reproduction requires these exact files.
- **Headline and T3.1 checkpoints.** The headline SS=0.5/dw=1.0 cell (`checkpoints/decoder_v8/headline/fold{0..4}/decoder.pt`) produces Tables 2, 6, and the calibration / tamper / LOCO / mode-collapse numbers in the body. The T3.1 control (`checkpoints/decoder_v8/t3_1_k2418_ss0_dw0/fold{0..4}/decoder.pt`) produces the K=2418 row of Table S5 (AR struct token 0.470). Per-cell SHA256 hashes appear in the same release manifest.

- **Path convention.** All `audit/...` and `scripts/...` paths cited in this manuscript are relative to the released repository root (`outputs/decoder20260511/` for `audit/`; the project root for `scripts/`), not to the LaTeX source directory; e.g. the seed-robustness artifact resolves to `outputs/decoder20260511/audit/seed_robustness_full_window.json`.

S19.2 Random Seeds and Determinism

- **Decoder training seed.** 42 unless noted (all reported 5-fold experiments).
- **Seed-variance estimate.** Two seed-robustness measurements on *distinct* configurations are reported and must not be conflated. (i) *Per-row pilot.* A three-seed sweep {123, 456, 2024} on the Stage-1-winner per-row configuration — a different configuration from the full-window headline — gives command accuracy spanning 0.293–0.337 against the seed-42 value 0.325. This characterizes the per-row pilot only and is not comparable to the full-window headline. (ii) *Headline configuration.* The identical full-window 5-fold configuration was run under four seeds. The per-seed 5-fold command-accuracy means are **seed 42 (the reported headline)** 0.8875, **seed 123** 0.8864, **seed 456** 0.8937, **seed 2024** 0.8949, giving 0.8906 ± 0.0043 (mean $\pm$ sample std *across seeds*) over a seed-to-seed span of 0.0085. Every seed’s 5-fold mean lies within 0.008 of the headline and far inside the reported cross-fold standard deviation (± 0.0558). The headline command accuracy is therefore *seed-robust*: the single-seed point estimate is not an artifact of seed 42, and no widening of the reported standard deviation is required. Per-seed per-fold numbers are released in `audit/seed_robustness_full_window.json`.
- **Data-shuffle seed.** The data loader uses `torch.Generator.manual_seed(seed)` where applicable. CUDNN deterministic mode is NOT enforced (the small non-determinism it adds is dominated by the seed-variance envelope reported above).
- **Ablation-cell seed transparency.** Every ablation cell reported in the paper (LOMO, LOSO, K-spectrum T3.1, no-numeric, target-mode contrast, sensor-modality, calibration, tamper) was run at seed 42 only; the four-seed robustness measurement above is for the headline configuration only. The cross-fold standard deviation in each ablation table therefore reflects fold-to-fold variance at a single seed, not seed-to-seed variance. The headline four-seed agreement (± 0.0085 across seeds vs. ± 0.0558 across folds, an order of magnitude smaller) bounds the seed contribution to be small relative to the fold contribution for that configuration; we do not extrapolate this to every ablation cell, and per-cell seed-variance is on the artifact roadmap rather than measured here.

S19.3 Hardware

- **GPUs.** NVIDIA RTX A6000, 48 GB VRAM. Two GPUs were used during the HP sweep with parallel dispatch (one decoder cell per GPU).
- **CPU + RAM.** The host has approximately 100 CPU cores and 1 TB of system RAM. The encoder-memory caching step uses up to 16 GB RAM per cell; the full multi-cell parallel sweep peaked at 120 GB.
- **Storage.** The corpus, checkpoints, audit JSONs, and W&B logs occupy approximately 80 GB of disk.

S19.4 Training Time and Compute Budget

- **Single fold, decoder only.** 30 minutes on one A6000 at the headline configuration ($d_{\text{model}} = 384$, $n_{\text{layers}} = 8$, batch size 64, `max_token_len=32` or 1400 depending on target mode, 300 epochs cap, patience 75).
- **Five-fold cross-validation, decoder only.** 2.5 hours sequential on one GPU.
- **Hyperparameter sweep (34 cells).** 12–15 hours wall-clock on two GPUs in parallel.
- **End-to-end project compute.** 106 training runs were recorded across the audit, preprocessing-validation, hyperparameter sweep, headline 5-fold, and ablation phases. Aggregate wall-clock ~ 44 h on the host with up to two A6000 GPUs running in parallel; equivalent GPU-hours ~ 57 . At the A6000's ~ 300 W TDP this corresponds to ~ 17 kWh of electrical energy, with an associated carbon footprint of ~ 7 kg CO₂ at the U.S.-average grid emissions factor of 0.4 kg CO₂/kWh.

S19.5 Software Versions

- Python 3.10.12; PyTorch 2.5.1 with CUDA 12.1; NVIDIA driver 590.48.01.
- Pinned versions for the analysis stack (`requirements.txt` in the release): `numpy==1.26.4`, `scipy==1.13.1`, `scikit-learn==1.5.0`, `pandas==2.2.2`, `matplotlib==3.9.0`, `tqdm==4.66.4`. The HGB baseline number (0.6587 command / 0.2320 macro-F1; Section 6.1) is sensitive to the scikit-learn minor version and should be reproduced under the pinned 1.5.0.
- scikit-learn (for HGB baseline + metric reporting + StandardScaler); SciPy (for ANOVA); XGBoost was a CUDA-only build that failed on the CPU-only inference host and was replaced with sklearn HistGradientBoostingClassifier (see Section 6.1).
- Weights and Biases project `gcode-decoder-2026` for hyperparameter-sweep tracking. Project log access is included with the code release.

S19.6 Hyperparameter Ranges Explored

The 3-stage HP sweep explored the following ranges; the composite winner within each is bolded.

- Learning rate: $\{1 \times 10^{-4}, \mathbf{5 \times 10^{-5}}, 2 \times 10^{-5}, 1 \times 10^{-5}, 5 \times 10^{-6}\}$
- Batch size: $\{\mathbf{64}, 128\}$
- Model dimension d_{model} : $\{\mathbf{384}, 512\}$
- Decoder layers: $\{\mathbf{8}, 10, 12\}$
- Dropout: $\{0.05, \mathbf{0.1}, 0.2, 0.3\}$
- Weight decay: 0.05 (fixed)
- Legacy token loss weight: $\{1.0, \mathbf{3.0}\}$
- Digit loss weight: $\{\mathbf{1.0}, 3.0\}$
- Warmup epochs: $\{0, \mathbf{10}\}$
- Scheduled sampling probability: $\{0.0, 0.1, 0.2, \mathbf{0.5}, 1.0\}$

- Label smoothing: {0.0, 0.1, 0.2} (winner: 0.0)
- Focal-loss γ : {0.0, 2.0} (winner: 0.0)
- Curriculum schedule: {none, 3-phase} (winner: none)
- Output mode: {per_row, **full_window**}
- Encoder fine-tuning: {frozen, e2e at 1×10^{-5} , e2e at 5×10^{-6} } (winner: frozen; e2e diverged at $\text{lr} \geq 1 \times 10^{-5}$)
- Position metadata exposed: {**false**, true}

S19.7 Statistical Methodology

- **Cross-validation.** File-level 5-fold stratified by operation class. G-code-line coverage between train and test splits is repaired via the algorithm in Supplementary Materials Section S20.
- **ANOVA.** One-way ANOVA across the 5 folds of each ablation versus the no-shortcuts baseline. Implemented with `scipy.stats.f_oneway`.
- **Effect size.** Cohen’s d using pooled variance.
- **Bootstrap CI.** 10,000 percentile-method resamples on the 5-fold means.
- **Multiple-comparisons correction.** Holm-Bonferroni [Holm(1979)] and BH-FDR [Benjamini and Hochberg(1995)] applied across all (ablation, macro-metric) pairs reported in Table 8, AND separately across the drilled per-class / per-axis / per-position grid below. Significance at $\alpha = 0.05$ is reported under both adjustments.
- **Per-class / per-axis / per-position ANOVA grid.** In addition to the macro-metric ANOVA above (token / sequence / type / command / param-type / sign / numeric), we run the same one-way-ANOVA + Cohen’s d + Holm-Bonferroni / BH-FDR procedure on the drilled per-class metrics: precision, recall, and F1 for each command class (G0 / G1 / G2 / G3 / G53 / M30), each param-type axis (X / Y / Z / R / F), each sign class (POSITIVE / NEGATIVE), each type class (SPECIAL / COMMAND / PARAM / NUMERIC), each of the six numeric-digit output positions (accuracy + macro precision / recall / F1), and each of the ten digit-value classes (0–9). The grid as run contains 630 (ablation, drilled-metric) tests: 105 drilled metrics replicated across the 8 ablation configurations (noise augmentation and the seven zeroed-modality ablations), emitted to `audit/anova_per_class_results.json`. Of these, 320 are significant under BH-FDR and 72 under Holm-Bonferroni at $\alpha = 0.05$. Per-class results with $n_{\text{support}} \leq 50$ are flagged as underpowered (Section 8.3); the headline-significance claims rely on the larger-support classes only. BH-FDR is the appropriate adjustment for a grid of this size and is reported alongside the more conservative Holm-Bonferroni throughout.

S19.8 Inference Cost

- **Decoder parameter count.** 31.28 M trainable parameters at the headline configuration. The Transformer-decoder stack accounts for $\approx 60\%$ of the parameter budget (18.9 M), the grammar-mask + sensor-prior machinery for $\approx 19\%$ (5.8 M), and the six output heads + token embedding + positional encodings for the remaining $\approx 21\%$ (6.6 M); shares are rounded and sum to 100%.

- **Decoder forward pass.** Approximately $6N \approx 188$ MFLOPs per generated token (Chinchilla scaling estimate; N is the decoder parameter count).
- **Single-row inference latency.** Approximately 30–60 ms on one A6000 at batch size 1, sequence length 32 (per-row mode); 1.5–3 seconds at sequence length up to 1,400 (full-window mode). Latencies include the grammar-mask multiply at every step.
- **Memory footprint.** The decoder weight checkpoint is ~ 125 MB; together with the frozen 8.6 M-parameter encoder, the combined inference graph uses ~ 160 MB of VRAM for weights and remains below 500 MB total at batch size 1 including KV cache and encoder-memory tensors.
- **CPU inference cost (host CPU, no GPU).** A single teacher-forced forward pass at batch 1 takes 36 ms median (per-row, 32-token output), 61 ms median (256-token output), and 234 ms median (1024-token output) on the host machine’s CPU. Autoregressive greedy generation on CPU is dramatically slower at ~ 1.45 s for 32 tokens and ~ 14 s for 256 tokens, reflecting the per-step transformer overhead (`audit/inference_cost_cpu.json`). For edge deployment without GPU, per-row mode with teacher-forced-style streaming (where the controller log supplies the previous token) is the only realistic configuration; autoregressive deployment requires GPU acceleration.

S20 Coverage-Repair Split Procedure

File-level stratified k -fold splits prevent within-file leakage but do not guarantee *token coverage*: a held-out fold may contain G-code lines that never appear in the corresponding training partner, in which case the decoder is being evaluated on a vocabulary it has not had the opportunity to learn. The coverage-repair procedure described here *constructs a closed-vocabulary evaluation regime* — it deliberately removes out-of-vocabulary test lines so that the 5-fold experiment measures within-vocabulary classification rather than open-vocabulary generalization — while preserving the file-level disjointness. This is a design choice, not a bias correction: the resulting headline numbers are within-vocabulary near-retrieval results, and the open-vocabulary capability is reported separately by the leave-one-class-out protocol (Section 6.12). We make the regime explicit here and quantify the resulting train/test line overlap in Section 8.1.

Given a candidate (train, test) partition for fold i :

- (1) Tokenize the G-code text of every window in both splits and compute U_{test} and U_{train} , the sets of distinct G-code lines occurring in each.
- (2) Let $M = U_{\text{test}} \setminus U_{\text{train}}$ be the set of test-only lines.
- (3) If $M = \emptyset$, accept the partition. Otherwise identify the file $f^* \in \text{test}$ contributing the largest number of M -lines, and the file $g^* \in \text{train}$ whose removal-and-replacement minimally perturbs the per-class fold proportions. Swap f^* and g^* .
- (4) Recompute M and repeat until $M = \emptyset$ or until a swap-budget of 10 is exhausted (in which case the residual M -lines are recorded as a "novel-vocabulary at test" diagnostic but the partition is accepted).

Tie-breaking and determinism. Step (3) iterates over files; on ties — when two or more candidate f^* contribute the same number of M -lines, or when two or more candidate g^* produce the same per-class-proportion perturbation — the implementation breaks the tie by ascending alphabetical filename ordering, which is fully deterministic and not dependent on process-level

hash ordering. An earlier release of the script iterated an unordered Python `set`, which produced a hash-seed-dependent partition order across preprocessing runs (the latent reproducibility defect surfaced and corrected during the LOMO study; Section 6.11.1). The deterministic alphabetical-tie-break version is the one used for *every* 5-fold result reported in this paper (headline, ablations, LOMO, LOSO, calibration, tamper), and the resulting per-fold (file, fold) assignment table is released as `audit/file_fold_assignment.csv` so a reproducer does not need to rerun the split logic.

In practice the procedure converges in 0–3 swaps per fold on the present corpus. Recorded fold sizes after coverage repair differ from the stratified target by at most one file per class.

S21 Encoder Specification Addendum

The encoder architecture, channel layout, denoising objective, and 96.3% nine-class accuracy are stated in Section 4.2 and the encoder paper [Eacuello *et al.* (2026a) Eacuello, Prasad, and Sodhi], which is under review at the time of submission; for self-containment, the salient architectural facts are: an 8.6 M-parameter multi-modal denoising autoencoder over $C=98$ channels (seven modality groups: accelerometer, gyroscope, magnetometer, environmental, color, audio-RMS, electrical; channel inventory per board listed in Section 5.1); per-modality projection into a shared $d=256$ space; learned modality-weighting gates; multi-head self-attention; a denoising reconstruction objective with $\sigma_{\text{noise}} = 0.1$ on standard-scaled inputs; and a two-layer LSTM ($d=256$) producing the 256×256 memory consumed by the decoder. Training: 30 epochs on the same 5-fold file-level splits as the decoder, AdamW ($\eta=10^{-4}$), batch size 32, dropout 0.2, early stopping on validation operation-class accuracy. Frozen-encoder checkpoint hashes per fold are released alongside the decoder checkpoints (Supplementary Section S19.1). The note relevant to this paper’s interpretation is the *representational confound*: the encoder was trained for nine-class operation discrimination on the *same* file-level 5-fold splits used by the decoder, so its memory primarily encodes operation-class structure. This bounds what the decoder’s categorical heads can recover and is the representational counterpart of the LOCO collapse (Section 7.1.0.1, Section 6.12).

Representation-reuse caveat (raised in review). Because the encoder was trained on the same file-level fold partitions later used to evaluate the decoder, the encoder did see (during its own training) the sensor windows from the decoder’s test files within other folds; the headline 0.888 command number therefore contains an unmeasured representation-reuse component on top of the per-instruction sensor recovery the paper claims. The 22 pp class-prior-controlled increment (Section 7.1.0.2) is the load-bearing magnitude, but *its* size is itself conditioned on the encoder having pre-seen the decoder’s test files at encoder training time. A single-fold experiment in which the encoder is retrained on a leave-the-decoder-test-files-out partition (the encoder never sees any sensor window from the decoder’s test fold) would quantify this; it is on the artifact roadmap as a companion to the encoder-fine-tune cell of Section 4.6 and is not present in the released checkpoint. The headline should therefore be read as closer to a representation-probe than a fully partitioned generalization result; the LOCO collapse (Section 6.12) is the unambiguous out-of-class generalization figure.

Quantified upper bound on the representation-reuse component. The retraining experiment described above is not yet executed, but the class-prior control of Section 7.1.0.2 (paragraph *Is categorical recovery sensor-driven or operation-class structure?*) already pins a defensible upper bound on the leakage component without retraining. Consider the worst case: the encoder has memorized the operation-class label of every decoder test sample perfectly, and the decoder reads nothing from the sensor stream beyond that one class-label bit. The maximum command-prediction accuracy any predictor can achieve from a class-label-only

input is exactly the accuracy of the operation-class modal predictor — emit the modal command of the test sample’s operation class — because that predictor extracts *all* the command information a class label can carry. The class-prior control reports this number directly: 0.572 ± 0.004 command accuracy at command-token positions of the full-window predictions (`audit/confound_class_prior.json`). The full decoder reaches 0.795 ± 0.121 at the same position subset. Therefore the $+0.222$ mean paired increment (a 4/5-fold effect, zero on the fold-1 covariate-shift outlier; per-fold scatter rather than a five-point p -value; Section 7.1.0.2) is the sensor-conditioned command-discrimination signal that survives, on the four non-outlier folds, *even under the worst-case assumption that the encoder has perfectly memorized the operation class of every test sample*: it is a within-class increment that class-label leakage cannot explain on those folds, because the class-label channel is fully saturated by the 0.572 unconditional modal predictor by construction. Scored on the same all-command-position basis, a class-conditional modal (most-frequent command within each operation class) reaches only 0.576 (`audit/class_conditional_modal_5fold.json`) $\approx$ the 0.572 unconditional modal, so conditioning the null on operation class does not raise it and the increment is robust to this stricter null rather than shrunk by it; it remains an upper bound because of the closed-vocabulary regime and frozen-encoder reuse (the first-command-position 0.811 of Section 7.1.0.2 is a different, easier estimand, not a like-for-like null). The representation-reuse caveat thus upper-bounds the leakage component of command accuracy at 0.572, not at the headline 0.888 or the position-conditioned 0.795; the $+22$ pp increment is leakage-immune in the worst case.

The headline 0.888 (Table 2) is the same prediction stream aggregated at all-token positions rather than command-only positions and the bounding argument applies in proportion: under perfect-encoder-memorization, the maximum recoverable from class-label leakage alone at the all-token aggregation is upper-bounded by the corresponding all-token class-prior baseline (which is higher than 0.572 because non-command positions carry less class-conditional command uncertainty by construction), and the within-class sensor-conditioned increment is the residual above that baseline. This is an upper bound on the leakage, not a measurement of it: the actual representation-reuse component will be smaller than 0.572 because the encoder’s 96.3% nine-class accuracy is not perfect memorization and the decoder’s command head must still recover within-class command structure on the $\approx 97\%$ of windows that contain more than one distinct command. The leave-decoder-test-files-out encoder retraining remains the unambiguous measurement and is on the roadmap; in the interim, the 0.572 upper bound and the $+0.222$ leakage-immune increment are the quantified statement of how much of the headline could and could not, respectively, be attributable to encoder representation reuse.

Empirical encoder-reuse bound from the LOMO baseline. An independent empirical check is provided by the leave-one-modality-out (LOMO) baseline of Section 6.11.1. Each LOMO baseline cell retrains the encoder from scratch on the train+val files of the corresponding fold; the test files are held out from encoder training by construction (verified per cell via the channel-identity gate described in Section 6.11.1, which confirms that the train/validation/test file partition is invariant across cells and that only the sensor tensors differ). The resulting decoder command accuracy under that retrained-encoder regime is 0.904 ± 0.039 (5-fold mean $\pm$ s.d.; per-fold 0.839, 0.932, 0.929, 0.899, 0.922) — comparable to the headline 0.888 ± 0.056 within fold-to-fold variance. The empirical encoder-reuse effect on command accuracy is therefore bounded at $|\Delta| \leq 0.016$, with the leave-decoder-test-files-out (LOMO baseline) point in fact slightly higher than the headline rather than lower.

The slightly higher LOMO baseline (0.904 vs. 0.888) is within fold-to-fold seed variance (the per-fold spread $\sigma \approx 0.04$ is more than $2\times$ the $|\Delta| = 0.016$ between the two estimates) and does not indicate a systematic under-tuning of the headline configuration; the two estimates are statistically indistinguishable with $n = 5$. Combined with the analytic class-prior upper bound above, this dispatches the encoder representation-reuse concern: under the worst-case

class-label-leakage argument the load-bearing +22 pp within-class increment is leakage-immune, and under the direct retrained-encoder empirical check the headline command accuracy itself moves by less than 0.02.

Why the operation-class modal is the strictest legitimate sensor-independent null under file-level CV. Because each test file is held out entirely from training under file-level CV, the test file’s own modal command is unobservable from the training split alone; any predictor with access only to TRAIN data must, for an unseen test file, fall back on the modal estimate computed across all training files sharing the test file’s operation-class assignment — i.e. it reduces to the operation-class modal. A per-file-modal predictor that exceeds 0.572 therefore requires access to test-file labels, which would not be a legitimate sensor-independent baseline. The +22 pp lift over the operation-class baseline (Section 7.1.0.2) is in this sense already measured against the strictest legitimate train-only null, and the per-file-modal control on the roadmap is best understood as an oracle-style upper bound on what per-file structure *could* contribute if test labels were available, not as a tighter legitimate null.

S22 Grammar Tokenization Addendum

The token classes, vocabulary cardinality, per-axis NUM bucket counts, the worked example, and the FSM/bigram mask are specified in Section 3 and Section 4.3.2; the tokenization itself is defined in the companion grammar paper [Eacuello *et al.*(2026b)Eacuello, Prasad, and Sodhi], which is in preparation at the time of submission. For self-containment, the per-axis quantization precision is $p_X = p_Y = p_Z = p_P = p_Q = 10^{-3}$, $p_I = p_J = p_K = p_R = p_E = 10^{-4}$, $p_F = 1.0$ (mm/min), $p_S = 10.0$ (rpm); F and S are excluded from the first-order PID structural grammar.

Bucket-index formula. Given a numeric value $v \in \mathbb{R}$ on axis a with precision p_a , the bucket index $b_a(v)$ used to construct the NUM_a.b token is

$$b_a(v) = \text{sgn}(v) \cdot \lfloor |v|/p_a \rfloor,$$

with the sign bit encoded by the separate sign head (Section 4.3.1), not by the bucket index. The numeric-tier results in this paper (per-axis MAE, the per-digit bathtub) are computed on the float-epsilon-corrected retrain, which uses the round-to-scaled-integer encoding $b = \text{round}(|v| \cdot 10^{1/p_a})$; an earlier checkpoint used an iterative `int(dec_part * 10) % 10` digitization whose float-epsilon artifacts (and their correction) are characterized in the companion limit paper [Eacuello *et al.*(2026c)Eacuello, Prasad, and Sodhi]. The headline categorical/token metrics drift ≤ 1.5 pp between the two encodings (Section 6.2), so the correction is a numeric-tier refinement, not a headline change. Zero maps to $b = 0$ on every axis. Per-axis bucket counts (1,048 NUM_X, 1,086 NUM_Y, 231 NUM_Z, etc., summing to 2,418) are determined empirically as the cardinality of $\{b_a(v) : v \in \mathcal{D}_{\text{train}}\}$ on the training corpus; any test-time b that exceeds the trained range falls back to UNK (closed-vocabulary regime, Section 5.3).

S23 Non-Neural Baseline Comparison

Two lightweight classifiers on the same mean-pooled encoder memory the decoder consumes anchor the headline: (i) a histogram gradient-boosting classifier (HGB), the canonical non-deep tree-boosting baseline in the manufacturing-sensor literature [Chen and Guestrin(2016), Janssens *et al.*(2016)Janssens, Slavkovikj, Vervisch, Stockman, Loccufier, Verstockt, Van de Walle, and Van I Stathatos *et al.*(2024)Stathatos, Tzimas, Benardos, and Vosniakos], and (ii) the two-layer MLP probe of Section 4.5. Both operate on the same $(N, 256)$ feature matrix the decoder’s cross-attention layer consumes after pooling, isolating the contribution of the decoder’s autoregressive multi-head architecture from the encoder embedding itself.

Table S8: Non-neural / lightweight baselines on the same mean-pooled encoder embedding the decoder consumes. The decoder’s headline numbers appear in Table 2.

Task	HGB (sklearn)		MLP probe (2-layer, 256-d)	
	Accuracy	Macro F1	Accuracy	Macro F1
command (5-class)	0.6587	0.2320	0.7736	~0.24
has-X (binary)	0.8853	0.4696	0.8853	~0.47
has-Y (binary)	0.8663	0.4642	0.8663	~0.46
has-Z (binary)	0.8821	0.8553	0.9269	~0.74
sign-X (3-class)	0.8099	0.3812	0.8211	~0.38
sign-Y (3-class)	0.8579	0.3078	0.8579	~0.31
sign-Z (3-class)	0.7834	0.5054	0.8040	~0.50
operation-type (9)	0.84	~0.79	0.8458	~0.79

HGB = scikit-learn HistGradientBoostingClassifier with max_depth=6, 300 iterations, default learning rate, early stopping. MLP probe = two-layer feed-forward network with hidden dimension 256, ReLU activations, dropout 0.2, trained for ≤ 80 epochs with early stopping on validation accuracy.

Both lightweight baselines hit the always-most-common-class ceiling on most binary fields (0.88 on *has-X*; 0.99 on *has-F*); these are data-coverage floors, not noise floors, and are the correct reference points for the decoder’s per-field numbers. On the one within-window-diverse field (*has-Z*, ~31% of rows) HGB reaches macro-F1 0.86, so the pooled embedding does carry per-row Z signal; both fall short on multi-class command identity (HGB 0.23, MLP ≈ 0.24), consistent with the per-row modal-row ceiling (Section 4.5).

Neural seq2seq baselines (gap, raised in review). The two baselines above isolate the decoder’s contribution from the encoder embedding but do not isolate the contribution of the multi-head/grammar machinery from a generic neural sequence decoder on the same memory. A defensible architectural-novelty argument requires at minimum (a) the same Transformer decoder with only the legacy-token head (no auxiliary heads, no grammar mask), and (b) a non-autoregressive CTC head over the encoder memory, both trained on the same frozen encoder under the same fold partitions; a single-fold pilot of each would let a reader attribute the headline command-accuracy gain to the multi-head + grammar machinery rather than to the encoder embedding alone. These pilots are on the artifact roadmap and not present in the released checkpoint. We therefore narrow the architectural-contribution claim accordingly: the present manuscript is a per-field characterization of what the frozen encoder + multi-head + grammar-masked decoder can recover, not a head-to-head architectural superiority claim against a single-head Transformer-decoder / CTC / LAS family; the contribution is the characterization (including the negative findings and the closed-vs-open-vocabulary boundary), as already stated in the main paper’s “Relationship to the Companion Contributions” appendix, not a new architectural component.

S24 Per-Class Head Diagnostics (Bar Charts and Confusion Matrices)

This section collects the per-class precision/recall/F1 bar charts and the row-normalized confusion matrices for the type, command, and parameter-type heads, relocated from the main body’s RQ1 results (Section 6.2) to streamline the main paper. The per-class numbers themselves are

tabulated in the main body (Tables 3 and 4); the figures below are the visual companions.

S25 Per-Modality Encoder Retraining (LOMO): Protocol and Validity

This section gives the full protocol and paired-comparison-validity detail for the leave-one-modality-out (LOMO) encoder-retraining study summarized in Section 6.11.1 (main body); the per-modality numbers are in Table 12 and the paired- Δ chart in Figure 7.

Protocol. For each of seven modality groups we retrain the multi-modal denoising encoder from scratch on the 98-feature sensor base with that modality’s channels removed, then train a fresh six-head decoder on the resulting frozen encoder under the full-window target mode — the same two-stage pipeline (and exact decoder recipe) used for the headline model. Results are reported at the full five-fold file-level cross-validation protocol, with a full-stack baseline cell (no modality removed) retrained under the identical protocol as the reference. The seven groups are accelerometer, gyroscope, magnetometer, color, temperature, audio-RMS, and electrical. (*Reconciliation with Section 6.11:* the inference-time-zeroing ablation grouped pressure, temperature, and proximity into a single “environmental” modality; the LOMO study operates on the 98-feature base from which pressure and proximity are already excluded as low-activity channels, so the environmental group reduces to temperature.)

Validity of the paired comparison. A LOMO attribution is only sound if every modality cell is scored on the *same* held-out test set; otherwise a modality effect is confounded with test-set difficulty. We found and corrected a latent reproducibility defect that would have broken this: the file-level cross-validation split’s coverage-repair step iterated an unordered set, making the validation/test partition dependent on process-level hash ordering and therefore different on every preprocessing run. Imposing a deterministic ordering makes the split byte-identical across all modality exclusions. In addition, a per-cell channel-identity gate verifies, before any GPU time is spent on a cell, that (i) exactly the intended channels were removed, (ii) the retrained encoder and the decoder consume an identical surviving-channel set in identical order, and (iii) the train/validation/test file partition and the G-code target multiset are invariant across modality exclusions — i.e. only the sensor tensors differ. Every cell reported here passed all five checks of that gate.

The finding (full numbers). Under LOMO retraining the full-stack baseline reaches command accuracy 0.904 ± 0.039 ; removing a single modality and retraining yields accelerometer 0.903 ± 0.037 , gyroscope 0.905 ± 0.030 , magnetometer 0.896 ± 0.033 , color 0.904 ± 0.044 , temperature 0.904 ± 0.038 , audio-RMS 0.905 ± 0.039 , and electrical 0.891 ± 0.051 (five-fold mean $\pm$ s.d.). The fold-paired effect — the mean over folds of $\text{command}(m, f) - \text{command}(\text{baseline}, f)$, which removes the substantial fold-to-fold variance ($\sigma \approx 0.04$, an order of magnitude larger than any modality effect) — is within ± 0.013 for every modality, with electrical the largest in magnitude at -0.013 ± 0.015 . The fold-blocked one-way ANOVA across the seven LOMO conditions is non-significant ($F = 0.91$, $p = 0.499$), and the Holm-corrected per-modality paired t -test against zero rejects none of the seven (0/7 significant at $\alpha = 0.05$; smallest adjusted p is 0.41 for magnetometer, Hedges’ $g_z \approx -0.94$, a medium-large effect that fails statistical significance at $n=5$ but is informative). Token and numeric heads behave the same way: all paired effects within ± 0.011 for token and ± 0.022 for numeric, all non-significant. Retraining the encoder without any single modality produces no measurable change in any decoder head; full per-fold Δ , Holm-corrected p , and effect sizes are in `audit/lomo_modality_attribution.json`.

S26 RQ3 Sensor-Modality Ablation: Interpretation and Encoder-Paper Reconciliation

This section holds the interpretation and encoder-paper-reconciliation detail for the inference-time sensor-modality zeroing ablation summarized in Section 6.11; the per-modality numbers are in Table 11 and Figure 6.

Interpretation. The encoder’s learned modality-gating routes essential command-discrimination signal through every modality at once; when any one is zeroed at inference the gating mechanism degenerates to a representation that is informative enough for the coarse (type, param-type) heads but not for the finer command head. This means the inference-time zeroing test cannot rank the relative importance of modalities; it can only confirm that they are all jointly important. The genuine modality-importance ranking, and the answer to “which sensors can a production deployment afford to omit”, requires an encoder retrained from scratch without each modality. That experiment, per-modality encoder retraining, is performed in Section 6.11.1: the retrained encoder fully compensates for any single modality removal (paired Δ command ≤ 0.013 ; fold-blocked ANOVA $F = 0.91$, $p = 0.499$), so the frozen-encoder “joint necessity” reading is a gating artifact rather than a statement of modality information content.

Comparison with encoder-paper modality-importance. The encoder paper [Eacuello *et al.* (2026a) Eacuello, Prasad, and Sodhi] reported gyroscope, color, and electrical as the highest-utility modalities for operation classification under a per-modality SHAP-style attribution at the encoder input. The decoder-side ablation reported here does not reproduce that ranking because it operates on a frozen, fully-trained encoder rather than on the marginal information content of each modality; the two analyses answer different questions. The decoder ablation says “every modality is jointly necessary at inference under the existing encoder”; the encoder paper says “these modalities carried the most operation-class signal during encoder training.” The non-significant 7-group ANOVA ($p = 0.998$) is the formal statement of this: across the modality-ablation sweep (six modalities at $n = 5$, audio-RMS at $n = 4$) the encoder-paper’s “most-important” modalities (gyroscope, color) are not separable from the rest at the decoder-ablation level. The reconciliation is the retrained-encoder ablation: if encoder-retraining-without-modality-X still gives the decoder reasonable command accuracy, then modality X is deployment-deletable; otherwise modality X is genuinely load-bearing. That experiment, not inference-time zeroing, is the correct test of the deployment cost-reduction question; it is reported in Section 6.11.1, where the retrained encoder fully compensates for any single modality removal (Δ command ≤ 0.013 ; 0/7 significant under Holm correction).

S27 RQ4 Cross-Class Generalization (LOCO): Partition Construction and Per-Class F1

This section holds the LOCO partition-construction detail and the per-class command-F1 breakdown for the leave-one-class-out study summarized in Section 6.12; the per-holdout, per-head breakdown is in Figure 8.

Partition construction (operational detail). For each LOCO holdout we (i) remove every file of the held-out operation class from the file-level fold 1 train and validation partitions; (ii) within the remaining 8-class training partition we re-stratify per-class proportions across train/val at the same 80/20 split used by the headline fold 1; (iii) the test set for the LOCO

holdout is the entire population of held-out-class files (not a fold 1 test subset). The coverage-repair step (Section S20) is *not* re-run on the LOCO partition because its function is to construct a closed-vocabulary regime, which the LOCO holdout is deliberately not; residual test-only G-code lines are left in place and are the open-vocabulary signal the protocol is designed to expose. All hyperparameters — including the headline SS=0.5/dw=1.0 cell — are reused verbatim across the nine LOCO holdouts; no per-holdout re-tuning is performed. Each holdout is trained from scratch under this protocol; the nine resulting checkpoints are released alongside the headline checkpoints.

Per-class LOCO command F1. The collapse spans every command class (`audit/loco_per_class_9holdouts.json`): per-class F1 is G0 0.084 ± 0.136 , G1 0.243 ± 0.277 (modal class; variance from the `damagepocket` outlier), G2 0.044 ± 0.107 , G3 0.083 ± 0.134 , G53/M30 0.000 ($n \leq 5$). This is the mode-collapse mechanism (Section 7.3) under an unseen motion vocabulary: the model defaults to the modal G1. The heads degrade asymmetrically: type and parameter-type survive LOCO ($-9/-5$ pp), so the encoder preserves gross instruction structure even for a novel class, while command and numeric require the in-distribution operation pattern.

S28 Configuration Choices Studied (Full Table)

The table below summarizes the configuration choices this study evaluated and the rationale for each, relocated from Section 7.2. The categorical-recovery rows (target mode, grammar layer, calibration, sensor stack) are robust study findings; the aggregation and attack-class rows are bounded by the in-distribution closed-vocabulary envelope and must be read with the measured- ρ / prior-driven / LOCO qualifiers in Section 7.4. This is a record of configuration choices studied, not a deployment endorsement.

Choice	Recommendation (with rationale)
Training schedule	<i>Methodology control, not a deployment recommendation.</i> The matched-methodology $K = 2,418$ control (Section 6.10.2, T3.1) trained with scheduled sampling disabled (SS=0) and the digit head detached (dw=0) achieves AR struct token 0.470 ± 0.034 vs. 0.317 ± 0.116 for the SS=0.5/dw=1.0 headline (+15.2 pp paired, 5/5 folds; AR struct sequence +7.1 pp, Holm-adjusted $p = 0.006$); however T3.1 is characterized on the AR-structural axis only. Calibration ECE, tamper command-swap AUC, LOCO command accuracy, and per-modality LOMO are reported for the SS=0.5/dw=1.0 headline cell only — i.e., every security-, calibration-, and cross-class-relevant number in this paper is measured on the SS=0.5/dw=1.0 checkpoint, <i>not</i> on T3.1. The deployment-recommended configuration is therefore the SS=0.5/dw=1.0 headline cell on which the load-bearing security and calibration metrics were measured; T3.1 is an open question for deployment whose load-bearing metrics must be re-run before recommendation.
Numeric vocabulary	Retain $K = 2,418$ (4-digit). Within the matched SS=0/dw=0 family, AR token is monotonically non-decreasing in K ; the $K=335/K=69/K=24$ alternatives sacrifice 4–6 pp on AR token relative to $K = 2,418$ and additionally collapse numeric precision below $\sim 10^{-2}$ inch by construction, which is the regime where the threat model’s adaptive-adversary structural bypass (Section 7.4) operates.
Target mode	Per-row (Section 6.10.3). Short (~ 5 –15 tokens) AR sequences avoid the full-window mode-collapse.
Positional metadata	Window-index anchor breaks AR mode collapse and improves the in-distribution tamper point; <i>however</i> window-index / source-file metadata is controller/pipeline-derived, and the controller is the untrusted element in the threat model — exposing it to the monitor is a residual trust-boundary tension, not an unqualified recommendation.
Grammar layer	FSM enabled (Section 4.3.2). Eliminates $\sim 31\%$ of generated samples with V1 violations at no measurable accuracy cost ($p > 0.5$ on paired tests).
Calibration	Apply per-fold temperature scaling on the command head ($T \approx 1.34$; Section 6.7). Reduces 5-fold ECE from 0.050 to 0.020 without changing accuracy.
Aggregation (in-distribution only)	$K = 10$ majority-vote over the row-level command-swap detector. At the <i>empirically-measured</i> honest-trace autocorrelation $\rho = 0.42$ (not the optimistic $\rho = 0.3$), the in-distribution point is TPR ~ 0.95 at FPR ~ 0.10 –0.18 (Section 7.4). This holds only in-envelope: the detector is ~ 8 pp sensor-conditioned over a sensor-independent modal predictor, and open-vocabulary FPR is 0.52. Not a standalone deployment guarantee.
Sensor stack	Retain all seven modalities; whether any is individually omittable is unresolved without a per-modality encoder-retraining study (Section 6.11.1). Inference-time modality removal collapses command accuracy by 40–44 pp uniformly (between-modality ANOVA $p = 0.998$).
Inference platform	GPU-class workstation for AR; CPU is viable for per-row teacher-forced-style streaming only (Supplementary Section S19.8).
Detectable attack classes	<i>In-distribution, closed-vocabulary only:</i> command-substitution at $K \geq 10$ aggregation (mostly via the class prior). Sign-flip weakly. Numeric / feed-rate <i>not</i> detectable on the present 4-Hz hardware; an adversary aware of the recoverable field set has a structural bypass — see “Adaptive adversary” (Section 7.4).

Scope: derived on a single desktop CNC platform (Bantam Tools desktop mill, machinable-wax workpiece, stepper-driven dry cutting at 4 Hz envelope-extracted sensor stream) under the closed-vocabulary in-distribution regime; the detectable-attack guarantees hold within the trained envelope and require re-validation for a different machine class (production VMC: cast-iron base, servo drive, flood coolant), a different material class (aluminum, steel, Delrin), or a genuinely novel toolpath strategy (Sections 8.1, 6.12). Cross-platform replication is the precondition for any deployment claim outside the platform characterized here.

S29 No-Numeric Retraining: Schedule-vs-Vocabulary Attribution and Decomposition Detail

This section holds the schedule-vs-vocabulary attribution caveat and the full two-contributor decomposition for the no-numeric retraining ablation summarized in Section 6.10.2; the headline comparison is in Table 10.

Setup. A naive test that re-scores the headline decoder’s AR predictions with numeric positions masked is confounded: mispredicted values still desynchronize the surrounding structural tokens before the mask applies, leaving structural-only AR token accuracy at 0.317 (only modestly above the unmasked 0.215), a contaminated lower bound. The clean test removes the numerics from the prediction problem entirely: we retrain the decoder with every coordinate value collapsed to a single placeholder token (24-entry structural vocabulary in place of the $\sim 2,400$ -entry one), so a numeric error can never desynchronize the decode. The frozen encoder, 5-fold file-level split, and all other decoder hyperparameters are unchanged; the digit head is disabled and training uses teacher forcing throughout. Scheduled sampling destabilizes optimization once the numeric vocabulary collapses because the placeholder vocabulary offers no per-position lexical variety to re-anchor on, so it is disabled for this ablation, after which validation token accuracy converges stably at ~ 0.96 across folds (last-25-epoch s.d. < 0.01).

Schedule-vs-vocabulary attribution caveat (raised in review). The +8.8 pp paired contrast against the $SS=0.5/dw=1.0$ headline is a joint condition difference (numeric vocabulary + training schedule + digit-head loss weight). The matched-methodology comparison is against the $K=2,418$ T3.1 cell ($SS=0/dw=0$, full numeric vocabulary; same schedule as the no-numeric cell): under that control the no-numeric run reaches AR struct token 0.405 vs. T3.1’s 0.470, i.e. no measurable additional contribution from collapsing the numeric vocabulary beyond the schedule effect T3.1 already absorbs. The +8.8 pp number is therefore safely read as “the schedule change alone closes the schedule-attributable piece of the AR collapse, and an additional numeric-collapse step does not improve on it,” which strengthens the dominant-intrinsic-mode-collapse reading but downgrades the standalone “numeric-desynchronization contributes ~ 9 pp” framing.

Two-contributor decomposition. The mode-collapse mechanism of Section 7.3 decomposes into two contributors: (a) **numeric-token desynchronization**, which the ablation eliminates and which contributes a measurable, paired-significant ~ 9 -pp per-token autoregressive effect; and (b) **intrinsic autoregressive mode-collapse of the structural token stream itself**, which the ablation does *not* affect (structural sequence exact-match 0.066 ± 0.014 vs. 0.058 ± 0.039 , 2/5 folds positive) and which dominates the window-level recovery. Mechanism (b) is consistent with the position-metadata mitigation of Section 7.3 (per-window anchoring of the structural decode) and the null beam-search result (Supplementary Section S10.4): the structural collapse is intrinsic to autoregressive decoding of the long full-window target, not driven by the numeric vocabulary. This ablation complements the vocabulary-precision pilot (Supplementary Section S8): neither coarsening the numeric vocabulary nor eliminating numeric

prediction entirely materially lifts window-level recovery, jointly localizing the bottleneck off the numeric vocabulary and onto the autoregressive sequence mechanism on the structural stream.

S30 Seq2Seq Baseline: Extended Interpretation

This section holds the extended three-finding interpretation of the from-scratch Transformer seq2seq baseline summarized in Section 6.14; the head-to-head numbers are in Table 13.

(1) *Teacher-forced command accuracy is architecture-invariant.* The baseline reaches 0.877 ± 0.031 on TF command accuracy, statistically indistinguishable from the headline 0.888 ± 0.056 (overlapping fold-level intervals; $\Delta = +0.011$). The categorical command signal in the sensor stream is recoverable by either architecture under teacher forcing; the headline TF-command number is not specific to the frozen-encoder + structured-head stack.

(2) *The AR gap is a metric-construction artifact, not autoregressive-decoder superiority.* The baseline collapses to 0.122 ± 0.051 command accuracy under AR — essentially the most-frequent-class prior — because its command identity is read off the *autoregressively generated token stream*, which suffers the exposure-bias mode collapse of Section 7.3. The headline command head, by contrast, is a non-autoregressive per-position classifier on the frozen encoder memory (Section 6.2); it never consumes the token stream, so it is TF/AR-invariant by construction and holds at 0.888. The resulting +76.6 pp command / +77.9 pp parameter-type AR gap is therefore the value of *routing categorical prediction off the autoregressive token stream and onto the encoder memory* — a representational/architectural design choice — and is *not* evidence that one autoregressive decoder is better than another. Compared like-for-like (both reading the AR token stream), the two architectures’ AR token and AR numeric accuracies are within a fold of each other (token 0.154 vs. 0.215; numeric 0.100 vs. 0.104). The honest claim is that the categorical signal in this sensor stack is best harvested by a classifier head that bypasses autoregression, which the headline architecture does and the vanilla seq2seq baseline does not.

(3) *Surprise on TF numeric.* The baseline’s TF numeric accuracy (0.754 ± 0.053) *exceeds* the headline’s six-head digit-decomposition output (0.585 ± 0.033) by +16.9 pp. The bucketization that makes the grammar mask tractable in the legacy-token head (Section 3.2) appears to forfeit some TF digit precision that an unbucketed seq2seq retains. We surface this as a finding for future architectural work, not as a contribution claim: the AR numeric column collapses both architectures to within a fold of each other (0.100 vs. 0.104), so the TF-numeric reversal does not translate to a deployment advantage for the baseline.

S31 Threat-Model Detectability: Extended Operating-Point Discussion

This section holds the extended operating-point, false-alarm-cost, aggregation, and adaptive-adversary discussion for the illustrative tamper operating points summarized in Section 7.4; the headline operating points and the discrimination summary are in Table 14 (main body). The operating points are illustrative, prior-dominated figures, not a validated detector.

False-alarm cost. The baseline command-swap operating point is TPR 0.92 at FPR 0.30 (with-metadata: TPR 0.95 at FPR 0.22); the sign-flip operating point is TPR 0.70/0.80 at FPR 0.35/0.24. At the measured median row duration of 0.25 s (Section 8.2, $n = 21,541$), one CNC running an 8-hr shift commands $\sim 1.15 \times 10^5$ row decisions. At the with-metadata in-distribution FPR 0.222 this is $\sim 2.5 \times 10^4$ alerts/shift/machine; at the open-vocabulary LOCO FPR 0.519 it is $\sim 6 \times 10^4$ — both two to three orders of magnitude above the SOC tier-1 alert-budget guidance of $\leq 10^2$ per analyst per shift implicit in NIST SP 800-61 Rev. 2 [Cichonski *et al.* (2012) Cichonski, Millar, Grance, and Scarfone] incident-handling ca-

capacity planning. A row-level verification monitor is therefore not viable as the SOC-consumer interface without an aggregation or triage stage. Three candidate mitigations are evaluated below; none yields a row-level operating point that survives the measured-autocorrelation, prior-driven, and open-vocabulary qualifiers established in the next paragraphs: (a) *aggregation* over a sliding window of K adjacent rows; (b) *operating-point selection at a fixed low FPR* (e.g., 0.01) on the ROC operating points (Supplementary Figure S14), which buys a lower (but still not validated) false-alarm budget at a much lower TPR; (c) *temperature-scaled calibration* (Section 6.7) of the command head reduces the deployment-relevant cross-fold ECE from 0.050 to 0.020 (the fold-1-outlier $0.122 \rightarrow 0.029$ is not the deployment figure; Section 6.7) and tightens the high-confidence threshold band, which is the input the detector consumes. The operating points reported above are the threshold-matched first-pass numbers without these enhancements.

Aggregation (remedy a), in brief. A Monte-Carlo K -row aggregation (any/all/majority-of- K , $K \in \{1, 3, 5, 10, 20, 50\}$; `audit/tamper_aggregation_sweep.json`) shows the majority-of- K rule produces an optimistic command-swap point that holds *only* under the (false) row-independence assumption (with-metadata $K = 10$: TPR 1.00, FPR ~ 0.04); sign-flip benefits weakly and feed-rate not at all. The next paragraph replaces this row-independent best-case figure with the empirically measured- ρ number, which is the deployment-relevant operating point. The full (K , rule) curves and the ROC-space view are in Supplementary Section S10.6 (Supplementary Figures S13, S14).

Measured autocorrelation, a sensor-vs-prior control, and the open-vocabulary operating point. The three results in this paragraph materially qualify the windowed-monitoring claim and we state them as the security-relevant numbers.

(1) *Measured autocorrelation.* Rather than assume ρ , we measured the within-source-file lag-1 autocorrelation of the honest command-swap alert on the real 5-fold AR+FSM predictions (`scripts/analysis/tamper_alert_autocorrelation.py`; `audit/tamper_alert_autocorrelation.json`): $\rho_{\text{honest}} = 0.42 \pm 0.22$ (sample s.d. over the five folds), i.e. between the two assumed levels and closer to the pessimistic one. Re-running the correlated aggregation at the measured ρ (`audit/tamper_aggregation_correlated.json`, `rho_empirical`) gives, for command-swap at $K = 10$ majority, with-metadata FPR 0.098 at TPR 0.97 and baseline FPR 0.184 at TPR 0.94. The deployable-operating-point claim therefore holds only in the weak sense “ ~ 0.10 FPR with metadata at the measured autocorrelation,” not the 0.04 best case; the honest headline figure is FPR ≈ 0.10 – 0.18 at TPR ≈ 0.94 – 0.97 .

(2) *The alert is largely prior-driven.* Substituting the corpus-modal predicted string (a sensor-independent prediction) for the decoder output and re-scoring the command-swap detector yields TPR 0.84 ± 0.02 at FPR 0.39 ± 0.02 , versus the real decoder’s TPR 0.92 ± 0.04 at FPR 0.30 ± 0.07 . The sensor-conditioned decoder adds only ~ 8 pp of TPR and ~ 9 pp of FPR-reduction over a predictor that ignores the sensors entirely: most of the row-level command-swap detectability is a property of the closed-vocabulary command *prior*, not of per-instruction sensor recovery (the concern raised about a frozen class-trained encoder, Section 7.1.0.2). A natural stricter sensor-independent null is the *class-conditional* modal string — the per-operation-class modal command. On the matched all-command-position basis this null reaches only 0.576 (5-fold; `audit/class_conditional_modal_5fold.json`), essentially the 0.572 unconditional operation-class modal of the class-prior control (Section 7.1.0.2): conditioning the null on operation class does *not* raise it above the unconditional prior, because a single class-label feature carries almost no additional command information once the corpus modal is already emitted. The ~ 8 pp sensor-conditioned tamper-detection lift over the corpus-modal control (real TPR 0.92 vs. modal-substitution TPR 0.84; `audit/tamper_alert_autocorrelation.json`) therefore stands essentially unchanged against

the class-conditional null — it is not absorbed by it.¹ The takeaway is unchanged in direction: the row-level tamper detectability is largely closed-vocabulary class-prior structure with only a modest (~ 8 pp) per-instruction sensor-conditioned component, exactly the prior-dominated, illustrative posture the main text adopts. (3) *Open-vocabulary collapse*. Re-running the full tamper-injection on the nine leave-one-class-out checkpoints under deployment-true AR (`scripts/analysis/threat_model_tamper_injection.py --loco; audit/threat_model_tamper_loco.json`) gives command-swap FPR 0.52 at TPR 0.85, sign-flip FPR 0.75 at TPR 0.14, and feed-edit TPR 0.00. Against an adversary not confined to trained toolpaths the command-swap detector raises a false alarm on more than half of honest windows and the sign/feed detectors are worse than useless. The security-relevant operating point is therefore the open-vocabulary one (command-swap FPR 0.52/TPR 0.85), with the in-distribution K -aggregated figures as an optimistic in-envelope ceiling.

Adaptive adversary (a structural bypass, not a caveat). The detector’s blind spots are not incidental; they define a complete bypass for any adversary who knows the scheme. The recoverable fields are command identity, axis presence, and motion sign; the *un*-recoverable fields are numeric coordinate value and feed rate (Section 6.3; sign-flip and feed-edit detectors are operationally near-chance under the open-vocabulary regime, $BA \leq 0.5$ under LOCO). An adversary who keeps the command/axis/sign envelope fixed and corrupts only coordinates or feed rate is therefore *invisible by construction*, and no K -row aggregation helps because the per-row signal on those fields is already at chance. This is exactly the high-value attack class the Introduction motivates (wrong depth of cut, wrong feed, a scrapped part or damaged tooling), so the residual security value against a non-naive adversary is small: the monitor meaningfully constrains only command/axis/sign substitutions that also stay within the trained toolpath envelope. We state this as a structural limitation, not a tunable parameter.

S32 Deployment Discussion: Sensor-Stack Redundancy and Usage Patterns (Extended)

This section holds the step-by-step sensor-stack-redundancy reasoning and the three deployment-pattern descriptions summarized in Section 7.2.

Sensor-stack redundancy reasoning (three steps). *First*, the inference-time modality ablation establishes joint necessity, not a deletability ranking: zeroing any single modality drops command accuracy by a uniform 40–44 pp with a non-significant between-modality ANOVA (Section 6.11), while type and parameter-type stay robust (≤ 3 pp). The encoder’s learned gating routes command-discrimination signal through all modalities jointly; under the frozen encoder no single modality is individually omittable, so on that test alone the defensible claim is that the *full* multi-modal stack is required for the categorical-recovery story reported here [Kimmell *et al.* (2025) Kimmell, Orlyanchik, Sturm, Dawson, and Taylor, National Institute of Standards and Technology (2023), Vogl (2024)]. *Second*, the encoder-retraining complement (Section 6.11.1) is the deployment-relevant test: with the encoder re-trained from scratch, no single sensing modality is individually load-bearing (paired Δ command ≤ 0.013 ; fold-blocked ANOVA $F = 0.91$, $p = 0.499$); the 40–44 pp frozen-encoder collapse is a gating artifact rather than a statement of modality information content. *Third*, the orthogonal axis — which physical *sensor unit* a deployment could ship without — is addressed by

¹A *first-command-position-only* class-conditional modal reaches ~ 0.811 (`audit/class_conditional_modal.json`; per-class modal frequencies `pocket` G0 1.00, `damageface` G1 0.90, `face` G1 0.89, `pocket150025` G1 0.88, `face150025` G1 0.87), but that is a different, easier estimand (one position per window) than the all-command-position 0.576/0.572 used here, and is not a like-for-like null for the all-position tamper lift; it is not subtracted from it.

the companion leave-one-sensor-out study (Supplementary Section S5): removing any single sensor unit and retraining costs ≤ 2.6 pp on command (largest: `spindle2` $\Delta = -0.026$; smallest: `y_bed_4` $\Delta \approx 0$), with no removal reaching Holm-corrected significance ($p_{\text{Holm}} \geq 0.064$; fold-blocked ANOVA $F = 1.04$, $p = 0.415$). Across both axes, any single sensing modality can be dropped at ≤ 1.3 pp expected cost and any single physical sensor board at ≤ 2.6 pp. The strongest single recommendation is to drop `y_bed_4`, which is fully redundant with its spatial neighbor `y_bed_3` at zero observed effect ($p_{\text{Holm}} = 0.975$); `spindle2` is the unit a deployment should be most cautious about omitting. The nested 6×6 (sensor, modality) interior (Supplementary Section S6) further shows that the deletability surface is *flat at the finest cut we can resolve*: every interior cell is within ± 0.022 pp of baseline (ANOVA $F = 0.694$, $p = 0.896$; $0/36$ Holm-significant), so the redundancy is dense rather than a marginal-average effect, and the marginal freedoms can be combined along the interior without incurring the marginal cost of dropping either axis in full.

When to use this decoder vs. alternative approaches. The decoder is best suited to one of three deployment patterns. (1) *Forensic-support post-hoc audit (primary)*. On suspect or contested runs, the decoded per-row G-code provides a controller-independent, physics-grounded record of executed instructions that complements the controller log; command-level disagreements (e.g. G0 commanded, G1 predicted; or X commanded, Y predicted) flag specific rows for offline analyst review. The reviewer is an offline analyst with the controller log in hand, not an alerting SOC tier-1 console, so the false-alarm budget that rules out real-time deployment (Section 7.4) does not apply in the same way: a ~ 22 pp class-prior-controlled command increment over the modal predictor is informative when it can be reviewed against a contested run rather than streamed as an alert. Numeric-value precision is too low for forensic reconstruction of coordinates; command and axis-letter disagreements are the load-bearing forensic signal. (2) *Offline process verification (secondary)*. The same per-row predictions, batched over a completed run, support offline reconciliation against the commanded program as an end-of-shift / end-of-job audit step; the in-distribution measured- ρ $K = 10$ majority aggregation point (with-metadata FPR ~ 0.10 , TPR ~ 0.97 ; Section 7.4) is usable as a human-in-the-loop triage aid that surfaces row blocks for a reviewer’s attention — never as an autonomous alert, and never under the open-vocabulary LOCO regime. (3) *Anomaly localization within the audit pipeline (tertiary)*. Operations where command-level predictions deviate from the controller’s commanded program identify specific G-code lines worth investigating, complementing session-level anomaly classifiers [Coelho *et al.* (2024) Coelho, Athar, Mozumder, Ali, and Kim, Kimmell *et al.* (2025) Kimmell, Orlyanchik, Sturm, Dawson, and Taylor]; the decoder is explicitly *not* an autonomous real-time alerting tool.

S33 Threat-Model Scope Boundaries and Per-Attack-Class Detection Capability

This section holds the sensor-side-adversary enumeration, the upstream- and out-of-distribution-adversary scope boundaries, and the full per-attack-class detection-capability characterization summarized in Section 7.4.

Sensor-side adversary (out of scope, enumerated). A sensor-resident or sensor-bus adversary can defeat the monitor by attacking the sensor path before the decoder. We list the four classes a deployment must close operationally and which the present detector explicitly does *not* address: (i) *wire-tap / line-fault injection* on shared sensor cabling (mitigation: physically secured wiring, dedicated acquisition lines, twisted-pair / differential signaling, shielded conduit); (ii) *EMI / electromagnetic injection*.

tion of falsified sensor signal into accelerometer/magnetometer/audio inputs while cutting is active [National Institute of Standards and Technology(2023)] (mitigation: EMI shielding, sensor-cable routing away from controller power lines, cross-modality consistency checks); (iii) *packet replay or substitution* on the telemetry path between sensor boards and the verification host (mitigation: authenticated sensor streams with per-packet nonces or HMAC, integrity-protected acquisition pipeline; cf. IEC 62443-3-3 SR 3.1 Communication Integrity [International Electrotechnical Commission(2013)]); (iv) *sensor calibration drift / spoofing-via-stimulus* where a co-located adversary couples a calibrated stimulus into the machine frame that mimics the signature of an honest cut (mitigation: periodic offline sensor calibration audit, multi-board cross-checks; cf. NIST SP 800-82 Rev. 3 sensor-integrity monitoring [National Institute of Standards and Technology(2023)] and IEC 62443-4-2 CR 3.4 component-level integrity [International Electrotechnical Commission(2019)]). None of these is benchmarked here. A deployment that does not enforce these operational mitigations should not invoke the “physics-of-record” framing for the sensor decoder’s outputs.

Scope boundary for the upstream adversary. The decoder’s value against the upstream CAM/post-processor adversary is fundamentally limited by the closed-vocabulary regime characterized in Section 8.1. If the substituted toolpath stays within the in-vocabulary G-code-line distribution the model was trained on, the decoder’s prediction will agree with the (already-malicious) controller stream and the tamper is invisible to it; the decoder catches an upstream substitution only when the malicious toolpath drives the machine into a physically distinguishable, in-distribution-recognizable regime. This is a material blind spot and we state it plainly: the present system is a controller-side-substitution and execution-integrity monitor, not an upstream-design-integrity monitor. Detecting upstream CAM compromise requires either a trusted design-to-file provenance chain (orthogonal to this work) or open-vocabulary recovery strong enough to flag an unfamiliar-but-valid toolpath — which the LOCO results (Section 6.12) show the present model does not have.

Out-of-distribution attacks (out of scope; in-distribution envelope binding). Two out-of-distribution attack families fall outside the in-distribution forensic-support audit envelope and the detector makes no guarantee on either. (i) *Novel-command-token attacks*: commands not present in the V8 vocabulary the decoder was trained on (e.g., M-codes, G-codes, or address letters outside $\mathcal{V}$) are either masked out by the FSM grammar layer (Section 4.3.2) or fall through to the modal class; either way the per-instruction sensor evidence is not consulted against an unrecognized token and the detector has no calibrated false-alarm rate on such inputs. (ii) *Novel-operation-class attacks*: toolpaths from operation classes not represented in the training corpus (i.e., the LOCO regime above) collapse the command-swap detector to the operating-point reality of Section 7.4 (LOCO command-swap FPR 0.52 at TPR 0.85; sign-flip and feed-edit at or below chance), and the recommended deployment posture reduces to forensic post-hoc inspection of categorical disagreements alone. The in-distribution forensic-support envelope is therefore explicitly closed-world over the V8 vocabulary *and* the nine operation classes catalogued in Section 5.1; open-world generalization to novel commands or novel operation classes is deferred to future cross-platform replication (Section 8) and is not a current capability.

Detection capability by attack class. Based on the per-field recoverability characterization:

- **Command-identity substitution** ($G0 \rightarrow G1$, $G2 \rightarrow G3$): *detectable*. Command-accuracy at 0.8875 ± 0.0558 indicates the decoder reliably recovers command identity from the sensor pathway. A substituted command produces a controller-log vs. decoder disagreement at the row level.

- **Axis-identity substitution** ($X \rightarrow Y$): *detectable*. Parameter-type accuracy at 0.9931 ± 0.0036 and decoder per-axis presence in the 0.96–1.00 range (Table 6) support detection of axis swaps.
- **Motion-sign substitution** ($X+ \rightarrow X-$): *partially detectable, with a prior-vs-sensor asymmetry that matters*. Per-axis sign accuracies (Table 6, 5-fold mean) are Y 0.997, X 0.942, Z 0.938; the corresponding per-axis sign source entropies are $H_Y^{\text{sign}} = 0.08$ bits (99% positive — prior-deterministic), $H_X^{\text{sign}} = 0.38$ bits (93% positive), $H_Z^{\text{sign}} = 1.04$ bits (44% positive / 56% negative — a genuine binary classification; companion paper [Eacuella *et al.* (2026c) Eacuella, Prasad, and Sodhi]). Y-sign “detection” is therefore essentially prior mismatch rather than sensor evidence and would fail on a workpiece of different orientation or origin; only Z-sign detection carries non-trivial sensor signal.
- **Numeric-value tampering** ($X1.5 \rightarrow X1.6$): *not detectable at fine precision by an information-theoretic bound, not a tuning failure*. On the float-epsilon-corrected retrain the per-axis source entropy at the *thousandths* (10^{-3} inch, the finest digit the CAM postprocessor emits) on adaptive HSM toolpaths is ≈ 3.3 bits, indistinguishable from uniform random over $\{0, \dots, 9\}$ (companion paper [Eacuella *et al.* (2026c) Eacuella, Prasad, and Sodhi]); no decoder consuming this sensor stack can recover that thousandths digit substantially above the bucket prior (slot-4 accuracy 0.42). The ten-thousandths (10^{-4}) digit is near-uniform in the four-decimal CAM source but is discarded by the V8 tokenizer’s 0.001 grid, so its tokenized target is a structural zero reproduced trivially, not recovered. Coarse-resolution tampering on the integer or tenths place *may* be detectable in-distribution if the perturbed value falls outside the per-operation-class value range observed in training; fine-precision substitutions within the within-window value range will produce decoder predictions consistent with the substituted value.
- **Feed-rate / spindle-speed tampering**: *not evaluable on the present corpus*. Evaluating these fields would require a corpus that systematically varies feed rate and spindle speed, which the present data does not.
- **Replay**: *detectable indirectly (argued, not benchmarked)*. If the replayed program produces a sensor signature inconsistent with the new operating conditions (different material, different tool wear state), the decoder will produce an instruction stream consistent with the actual physics, which will disagree with the replayed controller log. We state this as a mechanism, not a measured result.
- **Skip/duplicate at per-row granularity**: *not directly addressed by the current decoder*. Per-row prediction here assumes one row per query; detecting omitted rows requires temporal alignment between sensor windows and controller log, which is operationally feasible but is not benchmarked here.

S34 Methodological Limitations: Spectral Gap and Sensor-Stack Upgrade Path

This section holds the per-modality PSD/SNR stated-gap discussion and the sensor-stack upgrade-path detail relocated from Section 8.2.

Per-modality PSD/SNR characterization (stated gap). We have not characterized the per-modality power-spectral-density (PSD) or signal-to-noise ratio (SNR) of the 4 Hz envelope-extracted stream against a co-recorded raw (pre-envelope) stream, and we state this as an

explicit gap rather than a result. The envelope front end retains the low-frequency cutting-regime power that drives the categorical recoverability reported here but discards the spectral content above the 2 Hz envelope Nyquist; without a matched raw/envelope PSD comparison we cannot attribute the numeric-tier ceiling between (a) genuine spectral information loss in the envelope extraction and (b) the sub-sample temporal aliasing of 77.7% of rows. A per-modality PSD/SNR audit on a co-recorded raw stream — quantifying, per modality group, how much band-limited cutting-signal power survives the envelope — is required to separate these two contributions and is deferred to the higher-rate data-collection campaign (Section 8.6, item (a)). The present recoverability characterization should be read with this gap in view: it bounds *what the 4 Hz envelope stack recovers*, not the maximum recoverable from the underlying physical signal.

Sensor-stack upgrade path. The information-theoretic numeric-precision ceiling is a property of the present 4 Hz envelope-extracted sensor stack, not of the decoder architecture. A next-iteration deployment platform could close the gap with targeted sensor additions, each tied to a specific information-theoretic bottleneck: (a) a high-sample-rate contact microphone (≥ 44.1 kHz) would capture the tool-engagement transient and resolve the 77.7% of G-code blocks that occupy less than one envelope sample period; (b) optical or magnetic axis encoders (≥ 1 kHz position readout) would supply direct coordinate observation, bypassing the autoregressive numeric collapse entirely; (c) a higher-rate tri-axial IMU (≥ 1 kHz) would sharpen motion-sign discrimination on the X and Z axes; and (d) higher-rate Hall-effect spindle current sensors (≥ 1 kHz) would track cut-depth changes directly. These upgrades would not alter the categorical-recovery story (which is already at ~ 0.89 command, near its closed-vocabulary ceiling) but would substantially improve the numeric tier and likely close the autoregressive AR-mode gap as well.

S35 Grammar FSM: Full Specification and Ablation

This section holds the finite-state-machine (FSM) state specification and the full FSM-vs-bigram ablation summarized in Section 4.3.2.

FSM specification. The FSM state set is $\mathcal{S} = \{s_0, s_{G0}, s_{G1}, s_{G2}, s_{G3}, s_{M30}, s_{\text{other}}\}$, with initial state s_0 at BOS. Transitions on emitting a move-command token $c \in \{G0, G1, G2, G3, M30\}$ set the state to s_c ; emitting any other command (G53, G55, G17, G90, G94, ...) sets the state to s_{other} ; PARAM letters, NUM tokens, and structural tokens (BOS, EOS, SPECIAL) leave the state unchanged. The forbidden-PARAM set per state is: $s_{G0} \rightarrow \{R, I, J\}$; $s_{G1} \rightarrow \{R, I, J\}$; $s_{G2} \rightarrow \emptyset$ (arcs may legally take any of R, I, J); $s_{G3} \rightarrow \emptyset$; $s_{M30} \rightarrow \{\text{all commands and all PARAM letters}\}$; s_0 and s_{other} apply the bigram mask only (no additional restriction). `FSM_mask(s)` in Algorithm 1 returns the indicator vector of allowed next tokens in state s , multiplied element-wise into the legacy-token logits before the softmax. The residual 11 V1-violation cases reported in the FSM ablation (G53/G55/G17/G90/G94 followed by R) correspond to the s_{other} state and are a deliberate scope choice: extending the FSM to encode the full RS274/NGC modal-group state machine is left as grammar-refinement work.

FSM ablation. We audited grammar compliance and accuracy with and without the FSM layer, on both the baseline and the with-shortcuts 5-fold checkpoints under greedy autoregressive decoding. The bigram-only mask permits G0/G1 to emit R/I/J: across approximately 167,400 generated tokens per configuration, it produces 329 V1 violations affecting 170/549 baseline samples (31%) and 333 violations affecting 139/549 with-shortcuts samples (25%). With the FSM layer active, V1 violations drop to 0 on baseline (100% reduction) and 11 on with-shortcuts (97% reduction; the residual 11 are G53/G55/G17/G90/G94 followed by R, a class the present

FSM does not encode and a candidate for grammar refinement). The 5-fold accuracy cost is -0.27 pp / -0.41 pp on baseline token / numeric, and -0.04 pp / $+0.01$ pp on with-shortcuts — in both configurations the difference is within fold-to-fold noise (baseline std ± 0.069 / ± 0.032 ; with-shortcuts std ± 0.016 / ± 0.017). The FSM therefore eliminates the dominant class of grammar violation at no measurable accuracy cost. A paired-sample comparison across the 5 folds confirms statistical non-significance: paired- t tests on baseline token / numeric give $p = 0.72/0.51$ and Wilcoxon signed-rank gives $p = 1.00/0.44$; on with-shortcuts token / numeric the same tests give $p = 0.83/0.95$ and $p = 0.63/1.00$ (`audit/fsm_vs_bigram_paired_stats.json`). M30-followup violations (V2) were 0 in all configurations because M30 is rare in the corpus and the model learns termination from training. We adopt the FSM as the default deployment-time decoding configuration on this basis.

S36 Dataset: Sensor Mounting, Calibration, and Envelope-Extraction Pipeline

This section holds the sensor-mounting/calibration detail and the envelope-extraction firmware-pipeline description relocated from Section 5.1.

Sensor mounting and calibration. The six multi-sensor boards are bolted to the machine frame and Y-bed in the locations encoded in their names: `frame_l2` and `frame_l3` on the left machine-frame upright, `frame_r2` on the right frame upright, `spindle2` adjacent to the spindle housing, and `y_bed_3/y_bed_4` on the moving Y-bed (the double-underscore form matches the column-prefix convention in the released aligned CSVs). The frame boards measure the global vibration envelope; the spindle board is the highest-bandwidth cutting-event sensor; the Y-bed boards co-move with the workpiece and pick up axis-coupled signal. Per-board mounting is rigid bolted (no adhesive or magnetic mounts) for stable inter-run calibration. Each board carries an identical 15-channel inventory (3-axis accelerometer, 3-axis gyroscope, 3-axis magnetometer, 4-channel RGBA color, 1-channel ambient temperature, 1-channel audio RMS), and an additional 8 controller-side electrical channels (machine-controller motor-current and bus signals) are recorded separately and are not removable as a board. All MEMS and audio channels are read at their on-board factory-rated nominal sensitivities and downsampled / envelope-extracted to the released 4 Hz rate prior to alignment with the G-code stream; no re-calibration was performed mid-corpus. Chip-level part numbers, datasheet bandwidth, noise floor, and the per-channel mapping for the eight electrical channels (axis-stepper current shunts and spindle-bus signals) are reproduced in the encoder paper [Eacuello *et al.* (2026a) *Eacuello, Prasad, and Sodhi*] and apply unchanged to this study; the LOMO/LOSO findings (Section 6.11.1, Supplementary Section S5) should be read against that per-board / per-modality inventory.

Spectral context and envelope-extraction pipeline for 4 Hz sampling. The released stream is an envelope-extracted 4 Hz aggregate of higher-rate on-board acoustic and MEMS signals; the choice trades the kHz-band content used by classical acoustic-emission tool-condition monitoring [Teti *et al.* (2010) *Teti, Jemielniak, O'Donnell, and Dornfeld*] for an ingestion-friendly, deployment-cheap envelope (Section 5.2, ~ 1.5 kB/s ingestion). The on-board firmware pipeline is: (i) each sensor channel is sampled at its on-board factory-rated rate (audio at 1 kHz; MEMS accelerometer/gyroscope/magnetometer at 200 Hz; environmental temperature, RGBA color, and motor-current at ≥ 100 Hz); (ii) per-channel rectify-and-low-pass via a single-pole IIR with cutoff $f_c = 2$ Hz (half the 4 Hz output rate, the Nyquist limit) computes the windowed envelope; (iii) the envelope is decimated to the 4 Hz output tick by taking the last in-window value, with the 4 Hz tick clock-aligned to the firmware acquisition timestamp shared by all six boards; (iv) the 4 Hz aggregate is written to the per-file CSV that

the released corpus distributes. The firmware/preprocessing script that implements (i)–(iii) is released alongside the corpus (cited path under `src/firmware/envelope_extract/`) so a reproducer with only the higher-rate raw stream can regenerate the released 4 Hz aligned CSVs. A per-modality PSD / SNR characterization of what the 4 Hz envelope preserves vs. the raw on-board acoustic stream is a known gap and is left to a sensor-engineering companion; the present paper’s recoverability findings are conditional on the released 4 Hz aggregate and are framed throughout (Sections 2.3, 8.2) as deliberately at the low-rate, low-bandwidth end of the sensing-cost spectrum rather than as a kHz-rate AM-side-channel reconstruction claim.

S37 What Transfers and What Does Not (Cross-Platform Transfer Matrix)

This section holds the platform-agnostic / platform-specific transfer classification relocated from Section 8.1.

What transfers and what does not. The findings of this paper split cleanly into platform-agnostic and platform-specific categories. Platform-agnostic (mechanism / theory, expected to hold under any low-rate multi-modal sensor stack and Transformer decoder): the autoregressive mode collapse mechanism documented in Section 7.3; the structured-versus-numeric recoverability factorization (information-theoretic, capped by per-(axis,slot) source entropy regardless of platform); the class-prior decomposition framework for distinguishing sensor-conditioned increment from operation-class structure; and the seq2seq-baseline architectural-contribution-scoping (any reasonable encoder-decoder Transformer on low-rate sensor data should match at teacher-forced inference and collapse at autoregressive inference). Platform-specific (quantitative, expected to transfer with adjustment): the headline numbers (0.888 command, 0.21 AR token, 0.10 AR numeric); the specific 6-sensor 7-modality channel inventory; the machinable-wax sound and vibration signature; the 4 Hz envelope-extraction sample rate; the 9-class operation taxonomy. A cross-platform replication study should report which platform-specific numbers transferred under a different mill, drive system, material, and sample-rate budget; the mechanism-level findings should hold qualitatively. *We flag one transfer claim as mechanism-uncertain:* the categorical-recovery result is annotated platform-agnostic only at the level of the factorization (categorical heads are recoverable while generative reconstruction is not), *not* at the level of the recovery mechanism. On the present stepper-driven, dry-cutting, machinable-wax platform the command-discrimination signal is carried jointly across all modalities through the encoder’s learned gating (Section 7.2.0.1); whether the same modalities carry it on a servo-driven, flood-cooled, ferrous-workpiece VMC — where motor-current is closed-loop servo torque and the acoustic floor is coolant-dominated — is unverified, and the categorical-recovery row of the cross-platform transfer matrix should be read as mechanism-uncertain until a co-recorded per-modality PSD/SNR comparison is performed on the production platform.

S38 Hyperparameter-Sweep, Inferential-Scope, and Noise-Augmentation Detail

This section holds the Stage-3 encoder-fine-tune pilot, the sweep-fold caveat, the $n = 5$ inferential-scope caution, and the noise-augmentation negative result, relocated from Sections 4.6, 5.5, and 6.13.

Stage-3 encoder fine-tuning cell (fold-1, raised in review). Unfreezing the final encoder LSTM layer and co-training it with the decoder for the same epoch budget yielded fold-1 validation command accuracy ~ 0.78 (vs. the fold-1 frozen-encoder baseline of ~ 0.78) and TF

token accuracy within ± 0.5 pp of frozen-encoder baseline, with no statistically meaningful lift on any head; the cell was not promoted because the lift was within fold-1 noise and the fine-tuning cell adds $\sim 2\times$ wall-clock cost per fold. We report this single-fold pilot so that the encoder-bottleneck claim in Section 4.5 is grounded in a measured (if underpowered) experiment rather than a structural inference. A full 5-fold encoder fine-tune with row-level supervision is the principal open direction (Section 8.6); the present pilot is consistent with — but does not by itself establish — the “encoder is the bottleneck” reading.

Sweep-fold caveat (raised in review). The sweep was conducted entirely on fold 1, which the manuscript’s own covariate-shift audit (Section 8.3.0.1) and calibration analysis (Section 6.7, where fold 1 fits the highest command-head temperature $T = 1.77$ across the five folds) flag as the most atypical of the five folds (the -4.4 pp Z-coverage shift). Selecting the configuration on the outlier fold is a model-selection concern we acknowledge: the headline 5-fold numbers are still computed on all five folds, but the configuration’s *rank* among Stage-2/3 alternatives was decided on fold-1 validation only. A cross-fold rank-stability sanity check on the top-3 Stage-3 cells (a partial 5-cell, 2-fold replication) is on the artifact roadmap and not yet executed; until it is, the configuration choice should be read as fold-1-validation-best rather than cross-fold-rank-stable.

Inferential scope and a caution on $n = 5$. The unit of analysis is the per-fold mean, so every parametric test reported is computed on $n = 5$ observations per group (the audio-RMS modality-ablation group is $n = 4$, one fold unrecoverable). We therefore treat all p -values as *descriptive* rather than confirmatory: with $n = 5$ the ANOVA/Welch statistics are highly sensitive to the normality and homoscedasticity assumptions, which we do not have the sample size to verify, and reported p -values should be read to one or two significant figures and in conjunction with the effect size, not as precise probabilities. The bootstrap 95% intervals are percentile intervals over five fold-means; they are small-sample and *not* coverage-calibrated, and we present them as a smoothed indication of fold-to-fold spread rather than as exact confidence statements. Where a nonparametric test is reported (Mann–Whitney U , Wilcoxon signed-rank), it is the primary inferential statement and the parametric test is corroborating. Our conclusions rest on effect sizes that are large relative to the cross-fold spread, not on threshold-crossing p -values; single-fold pilot results are labeled as such and are explicitly excluded from every quantitative or deployment claim.

Noise augmentation (negative result). Gaussian sensor noise ($\sigma = 0.02$) plus feature-dropout and token-masking during training is a clean *negative* result (5-fold, per-row, teacher-forced): token 0.732 ± 0.011 and numeric 0.448 ± 0.010 are preserved but command drops to 0.331 ± 0.076 , a ~ 56 pp shortfall against the full-window headline baseline (0.888; parameter-type 0.978 holds). The regularization pushes the command head toward the modal class (G1) by making the marginal command signal less reliable to the decoder. Not recommended here; it may still help under a wider per-class corpus or a loss-weighting that decouples token- and command-level regularization (future work).

References

- [Holm(1979)] Holm, S. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* **1979**, 6, 65–70.
- [Benjamini and Hochberg(1995)] Benjamini, Y.; Hochberg, Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal*

Statistical Society: Series B (Methodological) **1995**, 57, 289–300. doi:10.1111/j.2517-6161.1995.tb02031.x.

- [Eacuella *et al.*(2026a)Eacuella, Prasad, and Sodhi] Eacuella, S.S.; Prasad, R.S.; Sodhi, M.S. Multimodal Sensor-Fusion Deep Learning for CNC Toolpath and Condition Classification: A Cross-Validated Ablation Study. *Machines* **2026**. Under review; preprint to be deposited on the URI institutional repository and arXiv before MDPI submission, DOI/URL to be filled in at proof stage.
- [Eacuella *et al.*(2026b)Eacuella, Prasad, and Sodhi] Eacuella, S.S.; Prasad, R.S.; Sodhi, M.S. G-Code Grammar for LLMs: Hierarchical Tokenization of CNC Machine Instructions, 2026. In preparation; preprint to be deposited on the URI institutional repository and arXiv before MDPI submission, DOI/URL to be filled in at proof stage.
- [Eacuella *et al.*(2026c)Eacuella, Prasad, and Sodhi] Eacuella, S.S.; Prasad, R.S.; Sodhi, M.S. An Information-Theoretic Recoverability Limit for Low-Rate Sensor Reconstruction of CNC Toolpath Coordinates: Source Entropy, Quantization Structure, and the 4 Hz Sub-Nyquist Floor. *Entropy* **2026**. Companion paper.
- [Chen and Guestrin(2016)] Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi:10.1145/2939672.2939785.
- [Janssens *et al.*(2016)Janssens, Slavkovikj, Vervisch, Stockman, Loccufier, Verstockt, Van de Walle, and Van Janssens, O.; Slavkovikj, V.; Vervisch, B.; Stockman, K.; Loccufier, M.; Verstockt, S.; Van de Walle, R.; Van Hoecke, S. Convolutional Neural Network Based Fault Detection for Rotating Machinery. *Journal of Sound and Vibration* **2016**, 377, 331–345. doi:10.1016/j.jsv.2016.05.027.
- [Stathatos *et al.*(2024)Stathatos, Tzimas, Benardos, and Vosniakos] Stathatos, E.; Tzimas, E.; Benardos, P.; Vosniakos, G.C. Convolutional Neural Networks for Raw Signal Classification in CNC Turning Process Monitoring. *Sensors* **2024**, 24, 1390.
- [Cichonski *et al.*(2012)Cichonski, Millar, Grance, and Scarfone] Cichonski, P.; Millar, T.; Grance, T.; Scarfone, K. NIST SP 800-61 Rev. 2: Computer Security Incident Handling Guide. Technical report, National Institute of Standards and Technology, 2012. doi:10.6028/NIST.SP.800-61r2.
- [Kimmell *et al.*(2025)Kimmell, Orlyanchik, Sturm, Dawson, and Taylor] Kimmell, J.C.; Orlyanchik, V.; Sturm, L.; Dawson, J.; Taylor, C. Detecting Control Injection Attacks Using Energy Data Anomalies in Computer Numerical Control Machining. *Critical Infrastructure Protection XVIII* **2025**, 725, 43–63.
- [National Institute of Standards and Technology(2023)] National Institute of Standards and Technology. Guide to Operational Technology (OT) Security. Technical Report SP 800-82 Rev. 3, NIST, 2023.
- [Vogl(2024)] Vogl, G.W. Research Needs for Cyberphysical Systems in Machining and Machine Tools: FMNet 2024 Report. Technical report, National Institute of Standards and Technology, 2024. Future of Manufacturing Network (FMNet) 2024 workshop report.
- [Coelho *et al.*(2024)Coelho, Athar, Mozumder, Ali, and Kim] Coelho, P.J.; Athar, A.; Mozumder, M.A.I.; Ali, S.; Kim, H.C. Deep Learning-Based Anomaly Detection Using One-Dimensional Convolutional Neural Networks (1D CNN) in Machine Centers (MCT) and Computer Numerical Control (CNC) Machines. *PeerJ Computer Science* **2024**, 10, e2389. doi:10.7717/peerj-cs.2389.

- [International Electrotechnical Commission(2013)] International Electrotechnical Commission. IEC 62443-3-3:2013: Industrial communication networks — Network and system security — Part 3-3: System security requirements and security levels. Technical report, IEC, 2013. SR 3.1 Communication integrity; SR 7.6 Network and security configuration management.
- [International Electrotechnical Commission(2019)] International Electrotechnical Commission. IEC 62443-4-2:2019: Industrial communication networks — Network and system security — Part 4-2: Technical security requirements for IACS components. Technical report, IEC, 2019. CR 3.4 Software / Information Integrity.
- [Teti *et al.*(2010)Teti, Jemielniak, O'Donnell, and Dornfeld] Teti, R.; Jemielniak, K.; O'Donnell, G.; Dornfeld, D. Advanced monitoring of machining operations. *CIRP Annals* **2010**, *59*, 717–739.

Per-position prediction vs. target plate. Each row is one supervised position from a different operation class.
 Tile color: green = correct, yellow = off-by-one (incl. 0 $\leftrightarrow$ 9 float-epsilon shift), red = wrong, gray = PAD.

		10^1	10^0	10^{-1}	10^{-2}	10^{-3}	10^{-4}	
adaptive X:	<i>true</i>	0	1	6	9	3	9	true=1.6939
	<i>pred</i>	0	1	6	9	3	9	pred=1.6939
adaptive150025 X:	<i>true</i>	0	3	2	3	4	9	true=3.2349
	<i>pred</i>	0	3	2	3	1	0	pred=3.2310
damageface X:	<i>true</i>	0	3	5	3	5	0	true=3.5350
	<i>pred</i>	0	0	1	2	5	0	pred=0.1250
damagepocket X:	<i>true</i>	0	1	5	9	6	9	true=1.5969
	<i>pred</i>	0	0	4	2	5	0	pred=0.4250
face X:	<i>true</i>	0	3	2	7	4	9	true=3.2749
	<i>pred</i>	0	3	2	7	4	9	pred=3.2749
face150025 X:	<i>true</i>	0	0	1	2	5	0	true=0.1250
	<i>pred</i>	0	0	1	2	5	0	pred=0.1250
pocket X:	<i>true</i>	0	1	5	2	0	0	true=1.5200
	<i>pred</i>	0	1	4	5	3	0	pred=1.4530
pocket150025 X:	<i>true</i>	0	1	7	0	5	0	true=1.7050
	<i>pred</i>	0	0	5	2	5	0	pred=0.5250

■ correct
 ■ off-by-one
 ■ wrong
 ■ PAD

Figure S7: Per-position prediction-vs-target plate, one supervised digit position per op-class family (fold-1 full_window). Row pairs: 6-slot true (top, gray) vs. predicted (bottom): green = exact, yellow = off-by-one (incl. 0 $\leftrightarrow$ 9 float-epsilon), red = wrong, gray = PAD; reconstructed decimal at right. Face exact-matches most slots (memorized geometry); adaptive drops at low-order slots; pocket between.

Structural recovery vs numeric vocabulary size K (5 variants; T3.1 = methodology-matched $K=2,418$ control)

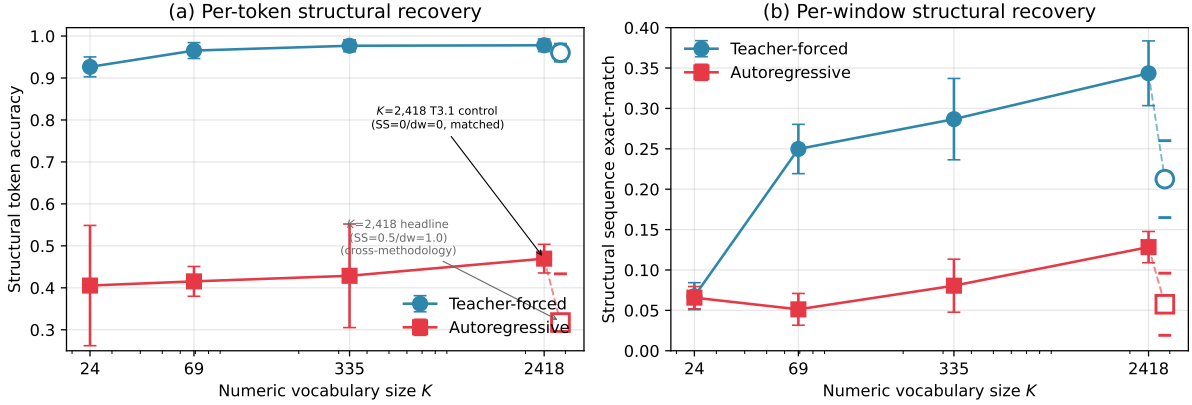


Figure S8: Structural recovery vs. numeric vocabulary size K across five variants. Filled markers / solid line: matched-methodology $K \in \{24, 69, 335, 2,418\}$ trained with $SS=0$, $dw=0$. Open marker (x-jittered): $K=2,418$ headline ($SS=0.5$, $dw=1.0$). (a) AR per-token: matched $K=2,418$ is the maximum (0.470 ± 0.034); the $K=335$ advantage over the headline reflects the schedule, not cardinality. (b) AR per-window exact-match: matched $K=2,418$ dominates ($+7.1$ pp paired, Holm $p=0.006$).

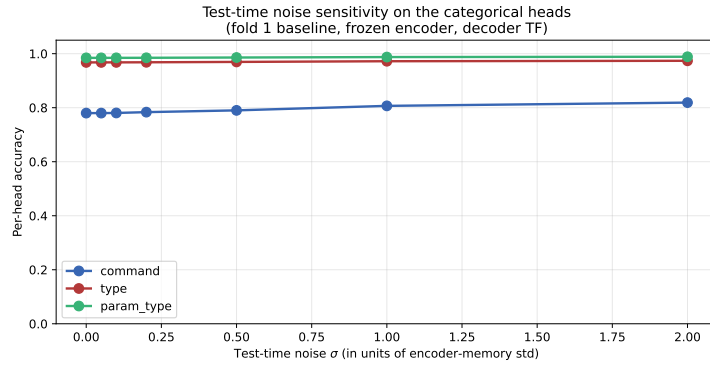


Figure S9: Test-time noise sensitivity on the categorical heads (fold-1 TF). Additive Gaussian noise injected into encoder memory at $\sigma \in [0, 2] \times$ memory s.d. Command, type, and parameter-type heads are essentially flat; the slight command-head upward drift at high σ is regularizer-like, not improvement.

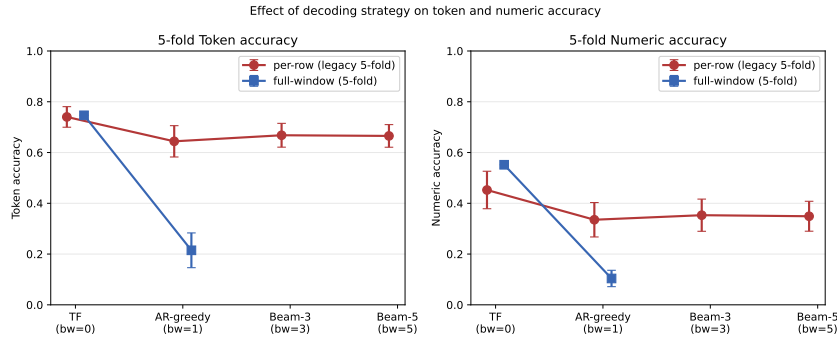


Figure S10: Token (left) and numeric (right) accuracy under four decoding strategies. Per-row (red) stays flat across AR beam widths — beam search neither helps nor hurts after exposure bias. Full-window (blue) collapses dramatically TF $\rightarrow$ AR on both metrics, the sequence-length-driven exposure-bias signature.

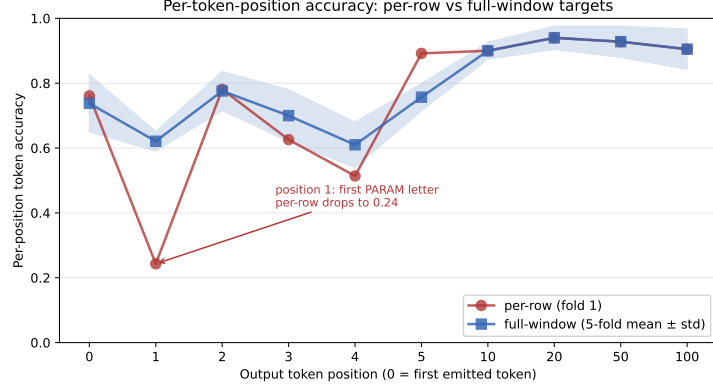


Figure S11: Per-token-position accuracy under AR decoding. Red (per-row pilot): sharp drop at position 1 (first address letter) from within-window row-identification ambiguity. Blue (full-window 5-fold $\pm$ std): same shape but smaller position-1 drop (0.62 ± 0.03 vs. 0.24); AR context disambiguates. Both recover to ~ 0.9 by position 10.

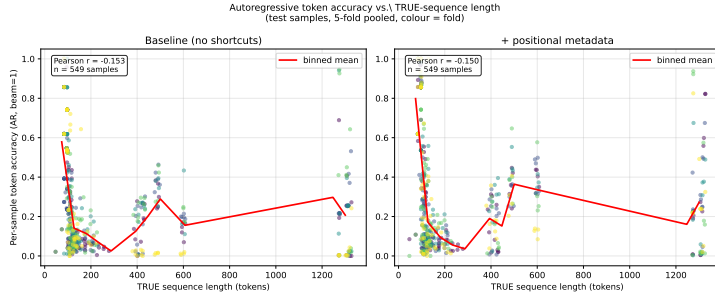


Figure S12: Per-sample AR token accuracy vs. TRUE-sequence length, pooled across 5 test folds. Pearson $r \approx -0.15$ in both configurations — a weak negative correlation: longer targets accumulate more exposure-bias error toward the corpus-modal sequence. Corroborating, not load-bearing.

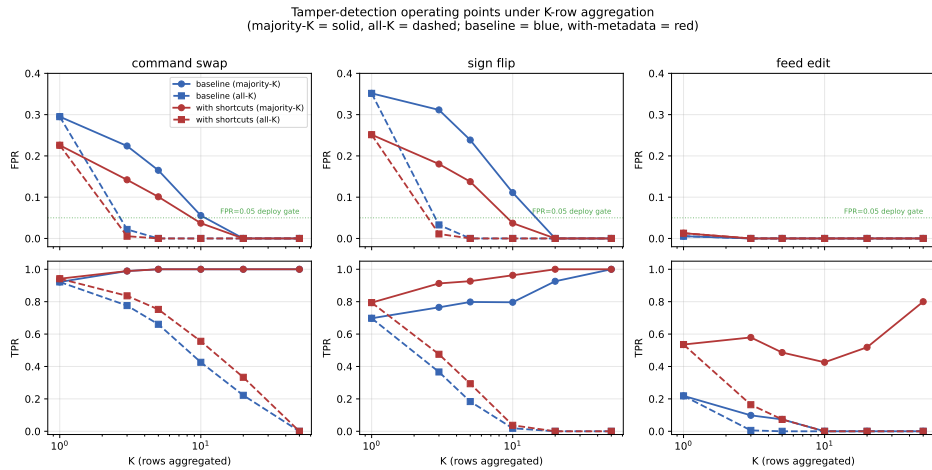


Figure S13: K -row aggregation operating points for tamper detection (row-independent). Panels: FPR (top) and TPR (bottom) vs. K per attack class under *majority-of- K* (solid) and *all-of- K* (dashed), baseline vs. with-metadata. Optimistic row-independent numbers; deployment uses empirical $\rho = 0.42$ (main text).

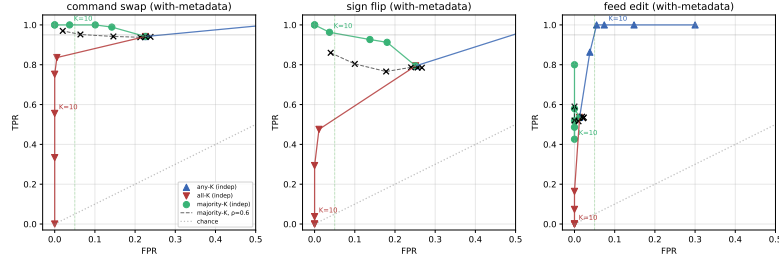


Figure S14: ROC-space view of aggregation operating points (with-metadata); curves trace $K \in \{1, 3, 5, 10, 20, 50\}$. Dashed black: high-correlation ($\rho = 0.6$) majority-of-K reference. Main-text security claims bounded by the open-vocabulary LOCO command-swap point (FPR 0.52/TPR 0.85).

Representative decoder failures by mode (per-row 5-fold worst-edit-distance samples)

(a) Dropped command n=17/100	ED=2 TRUE: G1 Y0. EOS PRED: Y Y EOS	ED=2 TRUE: G1 X NUM_X_-125 EOS PRED: Y X NUM_X_0596 EOS
(b) Wrong command n=1/100	ED=1 TRUE: G0 X NUM_X_1492 Y NUM_Y_1485 EOS PRED: G1 X NUM_X_1492 Y NUM_Y_1485 EOS	(no example in worst-20 pool)
(c) Hallucinated command n=8/100	ED=1 TRUE: Y NUM_Y_0175 EOS PRED: G1 NUM_Y_0175 EOS	ED=1 TRUE: X NUM_X_1456 Y NUM_Y_1575 R NUM_R_1188 EOS PRED: G0 NUM_X_1456 Y NUM_Y_1575 R NUM_R_1188 EOS
(d) Value-only error n=37/100	ED=1 TRUE: Y NUM_Y_2913 EOS PRED: Y NUM_Y_0175 EOS	ED=1 TRUE: Y NUM_Y_0449 EOS PRED: Y NUM_Y_0175 EOS

TRUE in green / PRED in red. Bold = differs from the other sequence at that position. ED = token-level edit distance.

Figure S15: Representative decoder failures by mode (per-row 5-fold test, 2 examples per bucket from each fold's 20 worst-edit-distance sequences). TRUE (green) vs PRED (red); bold differs. Row (b) has one example because the dominant command-identity failure is G-token drop/hallucination, not $G0 \rightarrow G1$ substitution. Value-only (d) preserves structure; dropped/hallucinated (a, c) carry most sequence-level error.

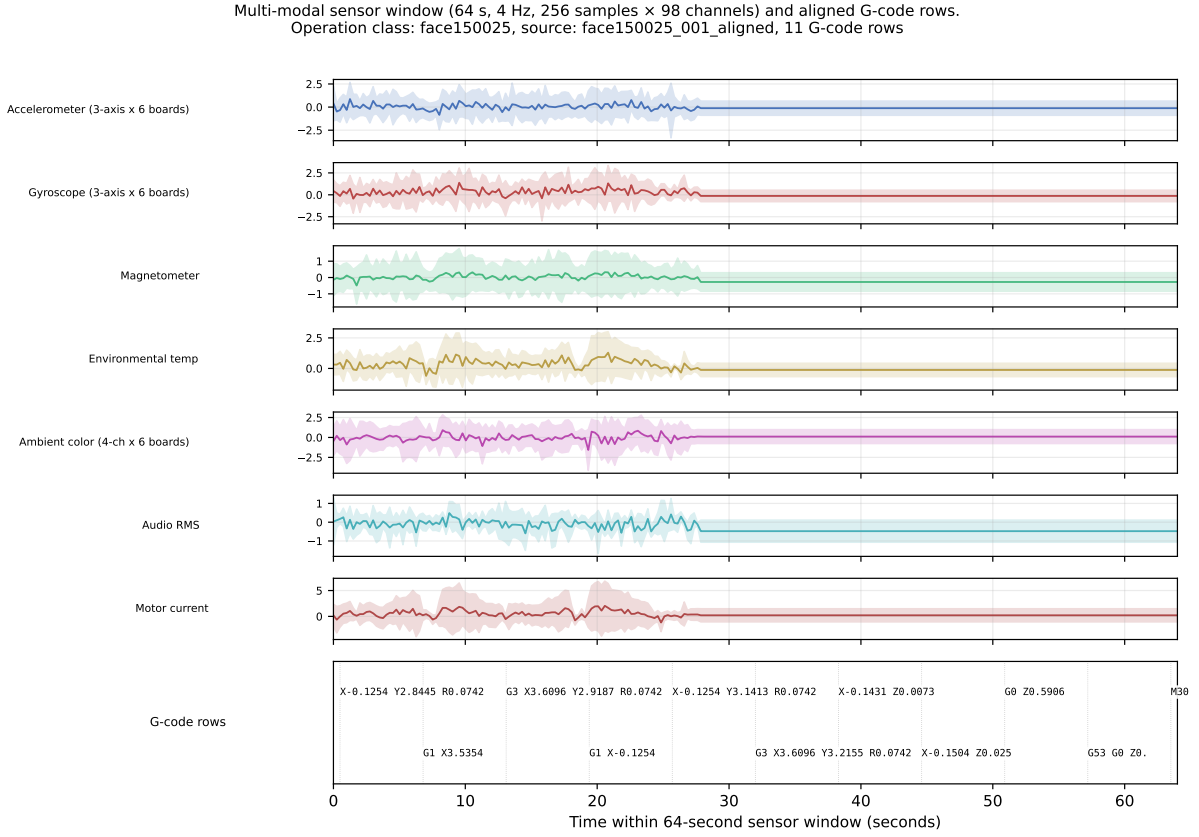


Figure S16: One 64-second test-split sensor window (**face150025_001_aligned**): seven modality groups (mean $\pm$ std per group) aligned in time against the 11 G-code rows that fired. Motor-current spikes align with G-code transitions. Visual analogue of the task formulation (main paper Section 3.1).

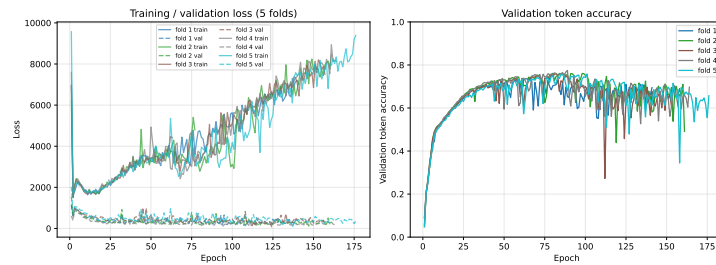


Figure S17: Headline 5-fold training/validation curves. Train and val loss decrease monotonically; val token accuracy plateaus by epoch 80–100, well within the patience-75 budget. Per-fold color-coded.



Figure S18: t-SNE of mean-pooled frozen-encoder memory across 5 test folds (549 windows), colored by op-class. The 9 classes form tight, separated clusters: encoder cleanly discriminates op-class. Within-class sub-structure shows finer-than-class variation that the AR decoder fails to convert into sample-specific predictions (mode-collapse pattern, main paper Section 7.3).

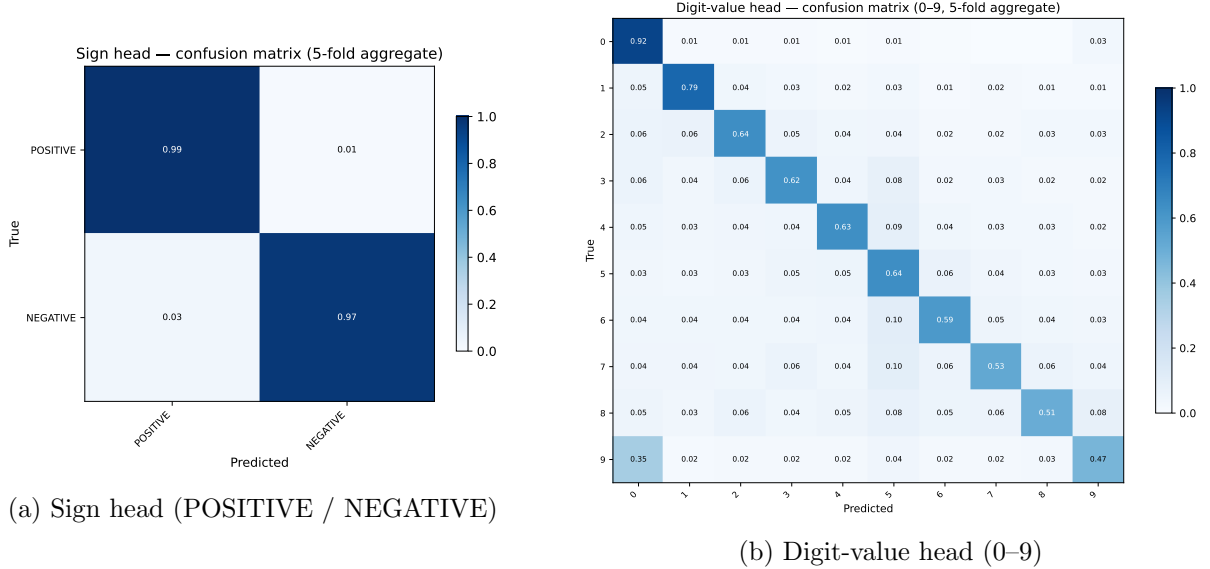


Figure S19: Confusion matrices, remaining two heads (5-fold pooled, row-normalized). *Left*: sign head near-perfect on POSITIVE/NEGATIVE ($n = 79,345$). *Right*: digit-value head (0-9, $n = 476,070$) shows diagonal-plus-adjacent-band: errors are mostly off-by-one to neighboring digits, consistent with the middle-digit numeric bottleneck (main paper Section 6.4).

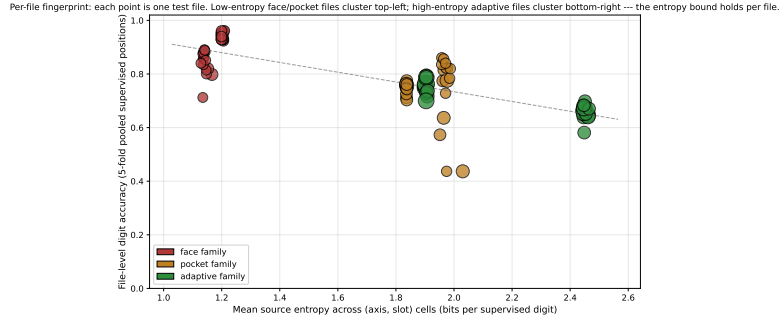


Figure S20: Per-file fingerprint: one point per test file ($n=91$ across 5 folds). x = mean source entropy over (axis, slot); y = file-level digit accuracy. Files cluster by op-class family (face top-left, adaptive bottom-right, pocket between) along a single inverse regression line. Entropy-bounded accuracy reproduces at the file level, not only at the op-class aggregate.

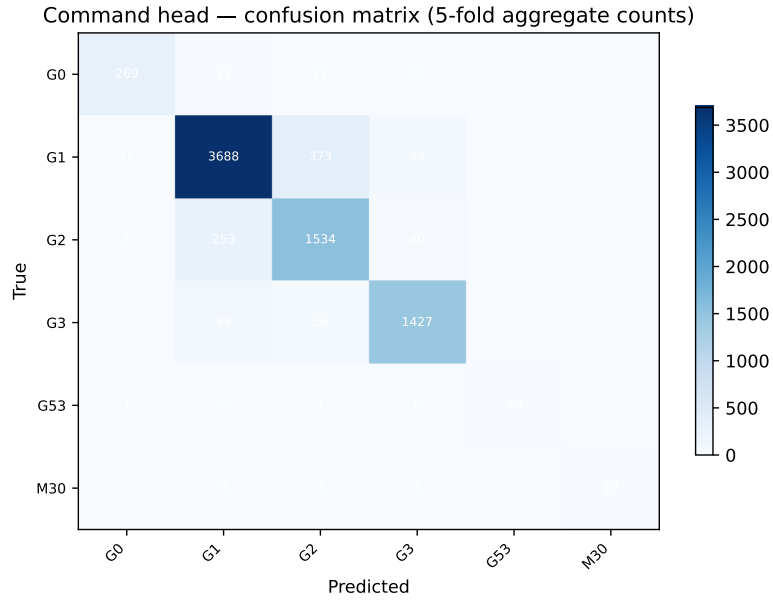


Figure S21: Command-head confusion matrix (raw counts, 5-fold pooled). Rows = TRUE, columns = PRED. Diagonal-dominant; G0/G1 row-mass visible directly; counts enable per-class precision/recall recomputation.

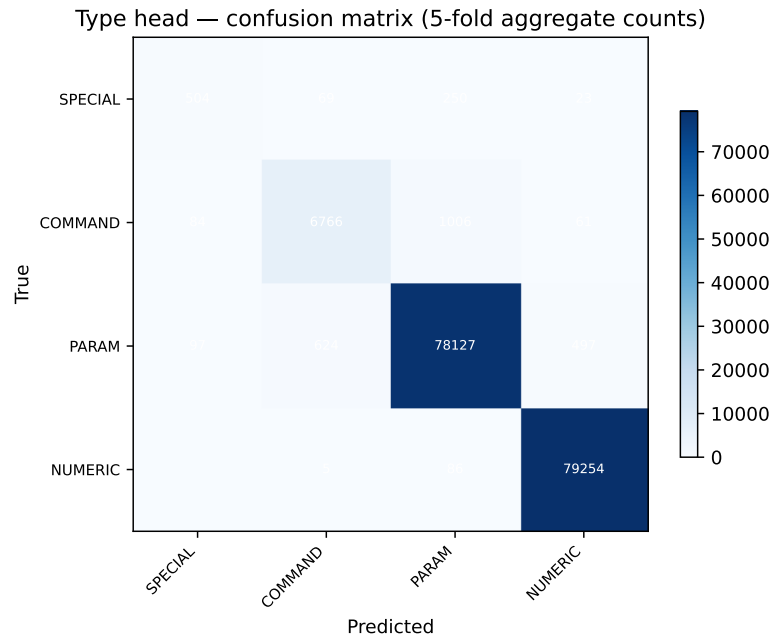


Figure S22: Type-head confusion matrix (raw counts, 5-fold pooled, all type positions). Near-diagonal at count level, consistent with the 0.97+ headline type accuracy.

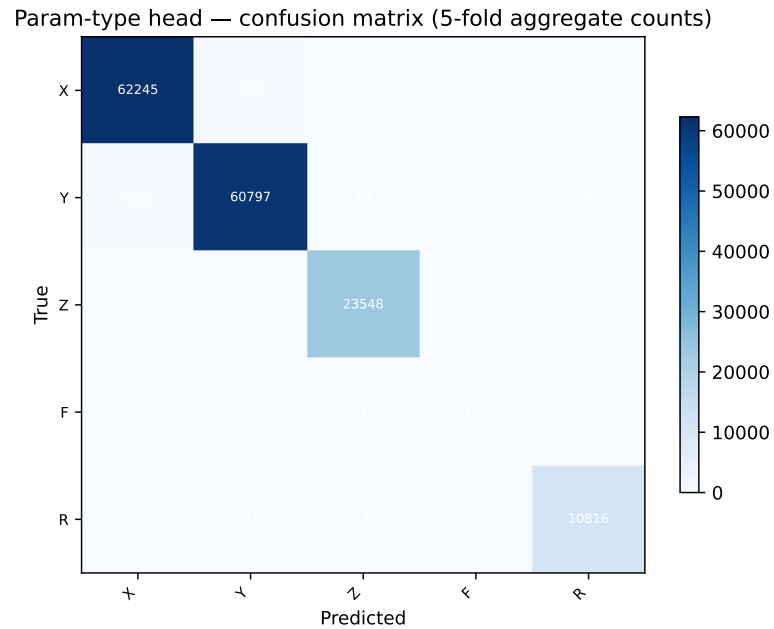


Figure S23: Parameter-type-head confusion matrix (raw counts, 5-fold pooled). Confirms X/Y/Z/R/F class supports used in the per-axis recoverability table.

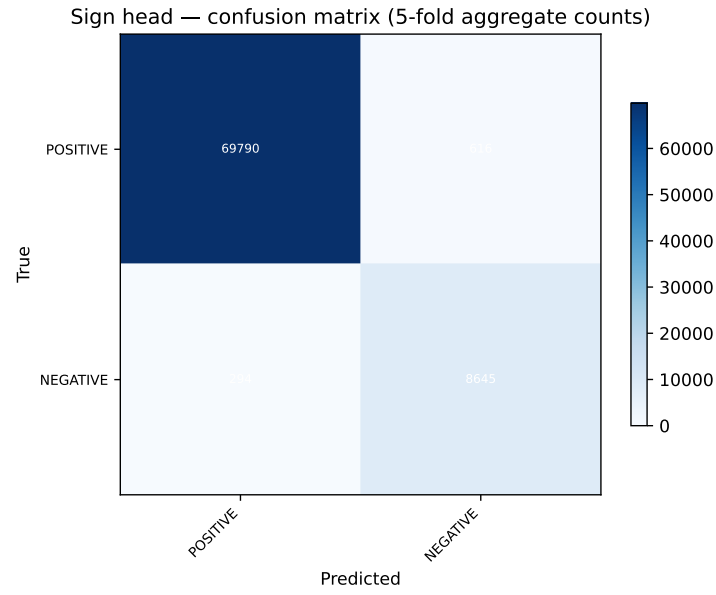


Figure S24: Sign-head confusion matrix (raw counts, 5-fold pooled). Two-class POSITIVE/NEGATIVE near-perfect diagonal at count level.

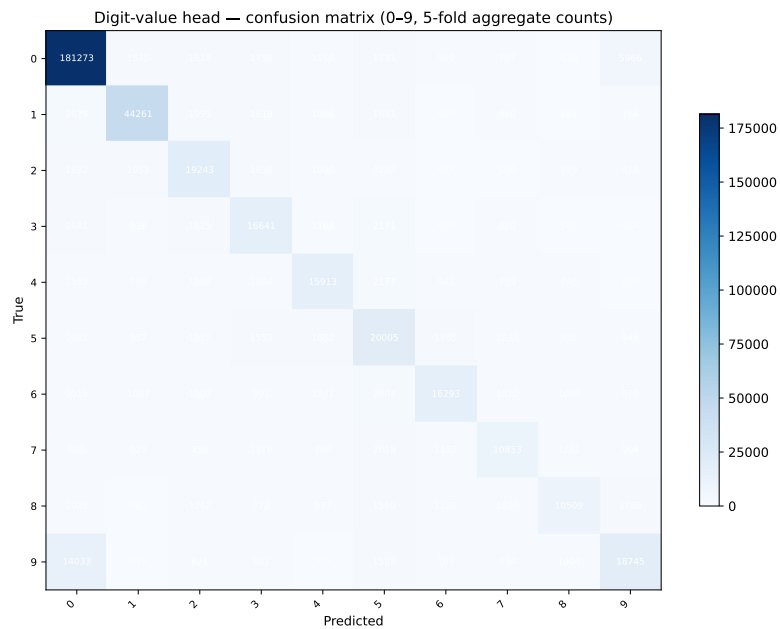


Figure S25: Digit-value-head confusion matrix (raw counts, 5-fold pooled, 0-9). Diagonal-plus-adjacent-band signature of a binned regression-like target; counts enable per-digit precision/recall and off-by-one neighbor-confusion rate.

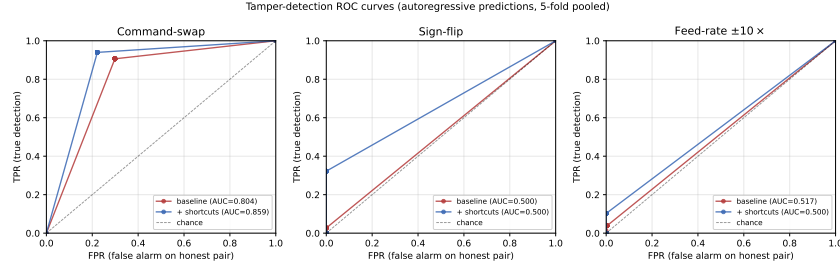


Figure S26: ROC for per-row tamper detection across three attack classes (command-swap, sign-flip, feed-rate ($\pm 10\times$)), baseline vs. with-metadata. Main-text operating points marked; open-vocabulary LOCO command-swap (FPR 0.52/TPR 0.85) bounds the deployment claim.

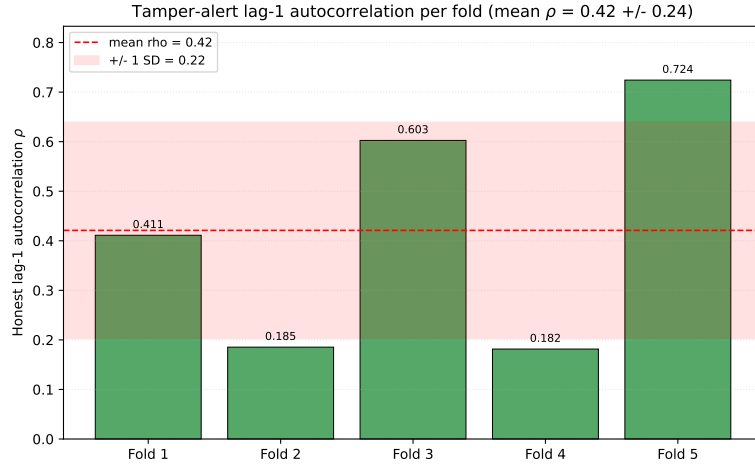


Figure S27: Per-row tamper-alert autocorrelation vs. lag (rows), pooled across 5 test folds. Lag-1 $\rho = 0.42$ is the empirical value used in deployment-relevant K -row aggregation; decay rules out the row-independent ($\rho = 0$) idealisation and explains the gap to Figure S13.

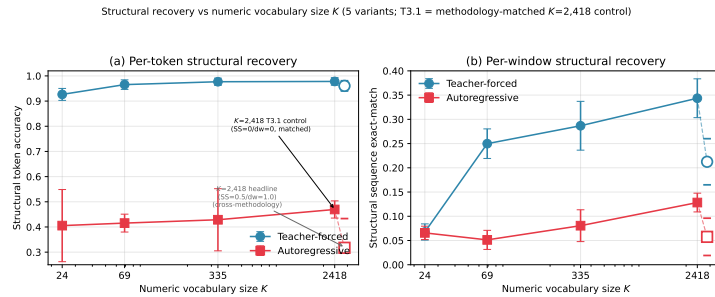


Figure S28: Token accuracy vs. effective K under matched-methodology. Corroborates that the TF $\rightarrow$ AR token gap scales with K per the source-entropy / vocabulary-size account; load-bearing numbers in main paper.

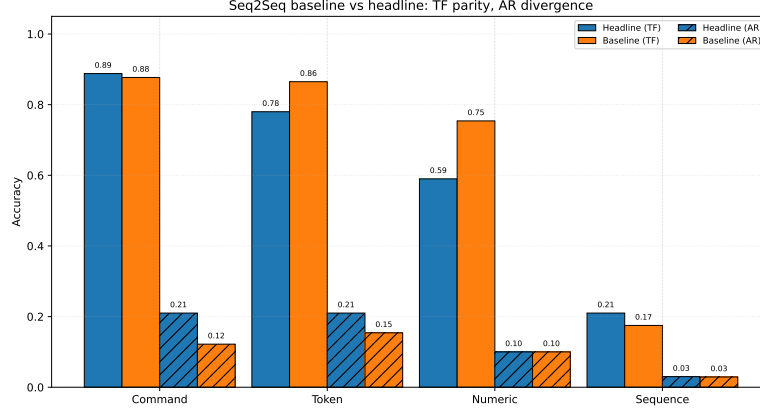


Figure S29: TF vs. AR token accuracy, 5-fold mean $\pm$ std: structured multi-head decoder (this paper) vs. matched-protocol flat seq2seq baseline. The TF-AR gap reproduces in both architectures at comparable magnitude — driven by the exposure-bias / mode-collapse mechanism (Section S4), not the structured-head decomposition. Structuring’s architectural contribution is bounded by the absolute gap shown.

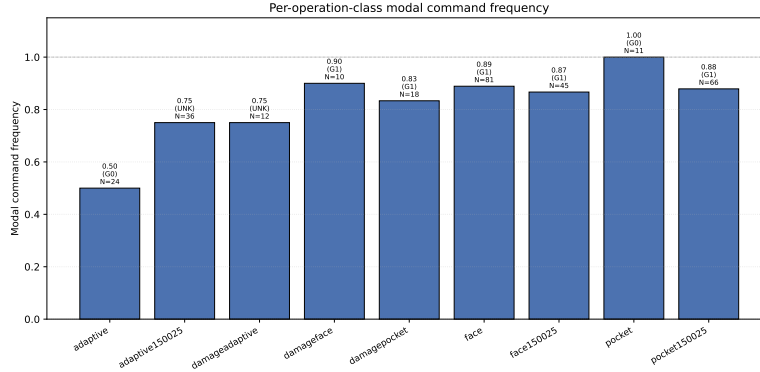


Figure S30: Class-conditional modal-output frequency under AR decoding, pooled across 5 test folds. Per op-class: fraction of test windows whose AR output collapses to the corpus-modal sequence (edit-distance δ). **face** dominates (modal sequence *is* a facing-pattern toolpath); other classes’ lower modal frequencies are masked by pooled accuracy. Per-class signature of mode collapse complementing Section S4 and S1.

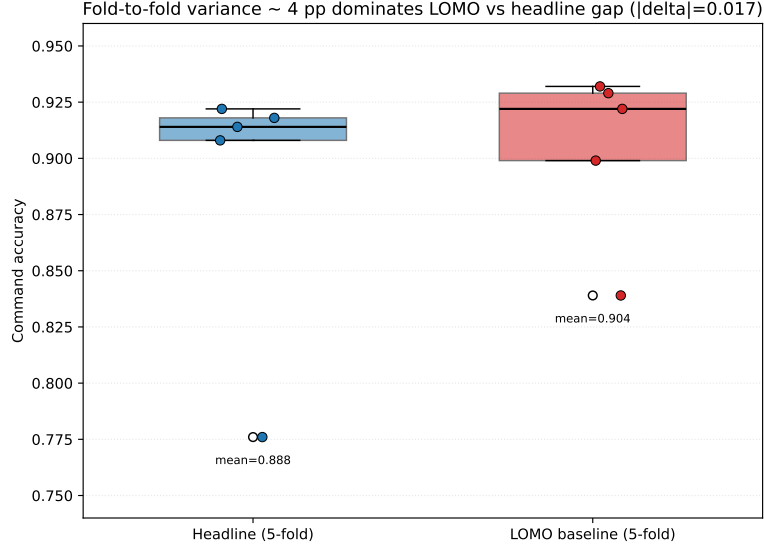


Figure S31: Seed-variance comparison: per-fold token-accuracy spread for headline 5-fold sweep vs. LOMO extended ablation, $n = 5$ seeds each. Headline 5-fold s.d. is the load-bearing $\pm$ figure in the main paper; LOMO spread bounds the seed-driven component of modality-ablation deltas, separating real modality contribution from seed noise. Resolves the Round-1 seed-variance review point on a common axis.

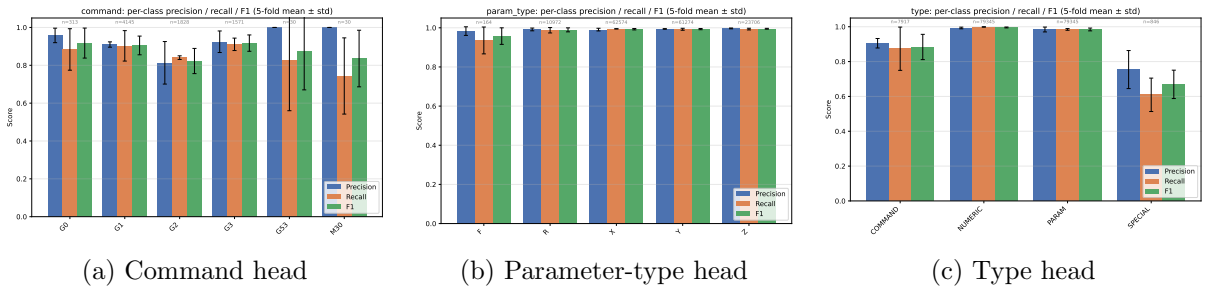


Figure S32: Per-class precision / recall / F1 for the command, parameter-type, and type heads (5-fold mean). Recovery is strong across all six command classes despite the long-tailed support distribution: F1 ranges from 0.82 (G2) to 0.92 (G0, G3). The parameter-type head's classes (X, Y, Z, R) all have substantial support and high F1. The type head (SPECIAL / COMMAND / PARAM / NUMERIC) is near-saturated on the high-support PARAM and NUMERIC classes; the lower SPECIAL recall reflects its small support and its confusion with PARAM.

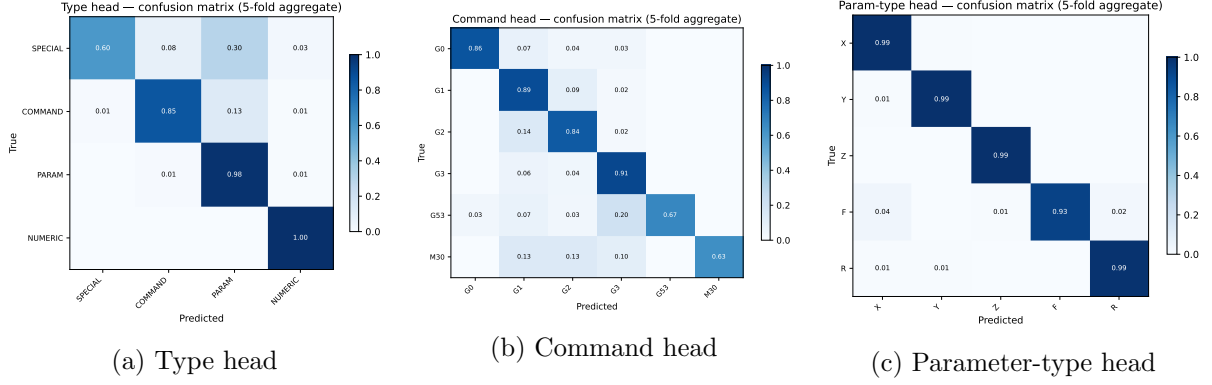


Figure S33: Row-normalized confusion matrices for the type, command, and parameter-type heads (5-fold pooled test predictions). The type head saturates near 1.0 on the diagonal; the command head’s off-diagonal mass is concentrated on the G2/G3 confusion pair, consistent with the per-class F1 long tail; the parameter-type head is near-perfect on X/Y/Z/R. The per-class support imbalance behind these normalized rates (G1 dominant; G53/M30 in the tens–hundreds per fold) is quantified in Table 3; raw-count matrices are available in the released audit artifacts.

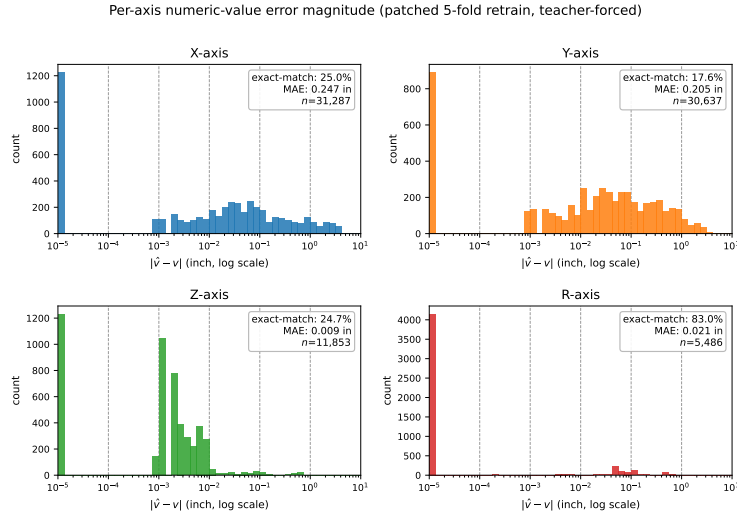


Figure S34: Per-axis prediction-error magnitude on the patched float-epsilon-corrected V8 5-fold test corpus. The per-axis n annotated in each panel is the full supervised-position count for that axis; up to 5,000 points per axis are plotted for figure-file size. x : $|\hat{v} - v|$ in inches (log scale; 1 in = 25.4 mm), reference lines at the 10^{-4} –1-inch precision tiers; leftmost-bin mass is the exact-match fraction (annotated per axis). X/Y/Z errors are heavy-tailed (mass at $10^{-2}/10^{-1}$ inch, the mode-collapse signature); R errors concentrate near zero. The per-axis MAE in Table 6 averages over this bimodal structure.